4x4 Systolic Array AI Accelerator: Complete RTL-to-GDSII Flow

Project Report

Hardware Implementation using SkyWater 130nm PDK

Ishaan Singhal
Delhi Technological University

Contents

Abstract

This report presents the complete design and physical implementation of a 4×4 systolic array-based matrix multiplication accelerator for artificial intelligence workloads. The accelerator is designed to perform INT8 precision matrix operations, which are fundamental to neural network inference.

The project encompasses the entire VLSI design flow from Register Transfer Level (RTL) design in SystemVerilog to the generation of a manufacturable GDSII layout file. The implementation uses the open-source OpenLane toolchain with the SkyWater 130nm Process Design Kit (PDK), demonstrating a complete tape-out ready design with zero DRC/LVS violations and timing closure at 100 MHz.

Keywords: Systolic Array, Matrix Multiplication, AI Accelerator, VLSI, RTL-to-GDS, OpenLane, Sky130 PDK

1 Introduction

1.1 Motivation: The Need for AI Accelerators

Artificial intelligence and deep learning have revolutionized modern computing, powering applications from autonomous vehicles to medical diagnosis. At the heart of these systems lies a computationally intensive operation: **tensor multiplication**. Neural networks process multi-dimensional data arrays (tensors) through billions of multiply-accumulate operations, creating a severe bottleneck for traditional CPU architectures.

Why Traditional CPUs Fail:

General-purpose CPUs are optimized for sequential instruction execution and branching logic, not for the massive parallel arithmetic required by deep learning workloads. Consider a modern Convolutional Neural Network (CNN) performing image classification:

- A single forward pass through ResNet-50 requires **4 billion MAC operations**
- Real-time video processing (30 fps) demands **120 billion MACs/second**
- A typical CPU can execute **10 billion MACs/second** across all cores
- **Result:** CPUs cannot meet real-time performance requirements

The Case for Hardware Acceleration:

Specialized **AI accelerators** address this performance gap through architectural innovations:

1. **Massive Parallelism:** Execute thousands of MAC operations simultaneously instead of sequentially
2. **Optimized Dataflow:** Minimize expensive DRAM accesses by reusing data within on-chip buffers
3. **Reduced Precision:** Use INT8 (8-bit integer) arithmetic instead of FP32, providing 4× memory bandwidth savings and 10× energy efficiency
4. **Fixed-Function Logic:** Eliminate instruction fetch/decode overhead with dedicated compute paths

Project Context:

This project implements the **hardware foundation** of a deep learning AI accelerator. While the design does not include trained neural network weights or inference software, it provides the complete **RTL-to-GDSII silicon implementation** of the computational engine that would execute tensor operations in a production AI system.

1.2 Understanding the Computing Problem

1.2.1 What is INT8 Precision?

INT8 (8-bit Integer) is a data format where numbers are represented using 8 bits, allowing values from -128 to +127 (signed) or 0 to 255 (unsigned).

Why INT8 for AI?

Research has shown that neural networks trained with 32-bit floating-point precision (FP32) can be **quantized** to 8-bit integers with minimal accuracy loss (¡1% for most models). This quantization provides:

Metric	FP32	INT8
Memory per value	4 bytes	1 byte
Memory bandwidth	1×	4× faster
Energy per MAC	100 pJ	10 pJ
Hardware area	1×	0.15× (6.7× smaller)

For inference (running trained models), INT8 precision is sufficient, making it the industry standard for edge AI devices.

1.2.2 The Convolution Operation

Neural networks perform **convolution**: sliding a small filter (kernel) across input data and computing weighted sums at each position.

Mathematical Definition:

For a 1D convolution with input **x** and weights **w**:

$$y_1 = w_1x_1 + w_2x_2 + w_3x_3 + \cdots + w_nx_n$$
$$y_2 = w_1x_2 + w_2x_3 + w_3x_4 + \cdots + w_nx_{n+1}$$

Each output y_i is a **dot product** of weights and input values. In 2D (images):

$$Y[i,j] = \sum_{m=0}^{M-1}\sum_{n=0}^{N-1} W[m,n] \cdot X[i+m, j+n] \tag{1}$$

Computational Breakdown:

A 3×3 convolution filter applied to a 224×224 image with 64 output channels requires:

$$\begin{aligned}\text{Total MACs} &= 224 \times 224 \times 3 \times 3 \times 64 \\ &= 28,901,376 \text{ operations per layer}\end{aligned} \tag{2}$$

Modern CNNs have 50-200 such layers, creating the need for specialized hardware.

1.3 Systolic Array Architecture

A **systolic array** is a network of processing elements (PEs) arranged in a grid, where data flows rhythmically through the structure like blood through the heart (hence "systolic"). Each PE performs one multiply-accumulate operation per clock cycle.

Key Characteristics:

- **Spatial Parallelism:** All 16 PEs in a 4×4 array operate simultaneously

- **Weight-Stationary Dataflow:** Filter weights remain fixed in PEs while input data streams through
- **Pipelined Execution:** Data propagates through the array in a wave-like pattern via delay registers
- **Scalability:** Larger arrays (8×8, 16×16, 256×256) provide proportionally higher throughput

Why Systolic Arrays for AI?

Google's TPU (Tensor Processing Unit), the most successful AI accelerator architecture, uses a 256×256 systolic array. The advantages:

1. **High Compute Density:** Simple, repeated PE structure maximizes silicon utilization
2. **Efficient Data Reuse:** Each input value is used by multiple PEs without refetching from memory
3. **Predictable Timing:** Regular structure simplifies physical design and timing closure
4. **Energy Efficiency:** Minimal control logic and short interconnects reduce power consumption

1.4 System Components Overview

1.4.1 Processing Element (PE)

The **PE** is the fundamental building block, performing one MAC operation:

$$\text{result} \leftarrow \text{result} + (a \times b)$$

Internal Structure:

- 8×8 multiplier: Computes product of two INT8 values
- 32-bit accumulator: Prevents overflow when summing many products
- Data forwarding: Passes input values to neighboring PEs

1.4.2 Systolic Array (4×4 Grid)

The array connects 16 PEs in a mesh topology:

- **Horizontal data flow:** Input activations stream left-to-right
- **Vertical data flow:** Filter weights stream top-to-bottom
- **Diagonal computation:** Results accumulate as data propagates through the grid

1.4.3 Input Buffers (SRAM)

On-chip **SRAM buffers** store input matrices temporarily, enabling:

- **Data reuse:** Avoid repeated DRAM accesses (100× slower than SRAM)
- **Burst transfers:** Fill buffers efficiently using AXI-Stream protocol
- **Systolic skewing:** Delay input rows/columns to create proper timing alignment

Each buffer holds 16 bytes (one 4×4 INT8 matrix).

1.4.4 Control Unit (FSM)

A finite state machine manages computation flow:

- **IDLE:** Wait for data load completion
- **COMPUTE:** Enable systolic array operation
- **DONE:** Signal output results are valid

1.4.5 AXI-Stream Interface

What is AXI?

AXI (Advanced eXtensible Interface) is ARM's industry-standard on-chip communication protocol, widely used in SoC designs. **AXI-Stream** is a simplified variant optimized for unidirectional data streaming.

Why AXI-Stream for This Design?

1. **Industry Standard:** Easy integration with ARM processors, DMAs, and other IP blocks
2. **Simple Handshaking:** TVALID/TREADY signals implement backpressure flow control
3. **Efficient Streaming:** No address overhead—data flows continuously once transfer starts
4. **Flexible Width:** Supports 8-bit (our case) to 1024-bit data buses

AXI-Stream Signals:

- `TVALID`: Source indicates valid data is available
- `TREADY`: Destination indicates readiness to accept data
- `TDATA[7:0]`: 8-bit data payload (one INT8 value per cycle)

Transfer occurs only when both TVALID and TREADY are high, preventing data loss if the buffer is full.

1.5 Project Objectives

This project demonstrates **full-stack hardware design** from algorithmic specification to manufacturable silicon:

1. **Frontend Design (RTL):**
   - Design a 4×4 systolic array in SystemVerilog
   - Implement AXI-Stream slave interfaces for data ingress
   - Integrate SRAM buffers with delay registers for systolic skewing
   - Verify functionality through comprehensive testbenches
2. **Backend Design (RTL-to-GDSII):**
   - Synthesize RTL to gate-level netlist using SkyWater 130nm PDK
   - Complete physical design: floorplanning, placement, CTS, routing
   - Achieve timing closure at 100 MHz operating frequency
   - Generate DRC/LVS clean GDSII layout ready for fabrication
3. **Verification and Documentation:**
   - Perform static timing analysis with extracted parasitics
   - Measure power consumption and energy efficiency
   - Document the complete design methodology and results
   - Demonstrate tape-out readiness through sign-off reports

2 Design Specification and Architecture

2.1 System Overview

The AI matrix multiplication accelerator consists of a 4×4 grid of processing elements connected in a systolic topology. The system accepts two 4×4 matrices as input through AXI-Stream interfaces and produces a 4×4 result matrix.

Top-Level Block Diagram:

Top-level architecture showing: AXI-Stream Interfaces → Input Buffers → Delay Registers → 4×4 Systolic Array → Output Results

Figure 1: System-level block diagram of the AI accelerator

2.2 Design Parameters

Table 1: Design Specifications

Parameter	Value
Array Configuration	4×4 (16 Processing Elements)
Data Precision	INT8 (8-bit input, 32-bit accumulator)
Target Frequency	100 MHz (10 ns period)
Technology Node	SkyWater 130 nm
Interface Protocol	AXI-Stream (slave)
Peak Throughput	16 MAC ops/cycle = 1.6 GOPS
Input Buffer Size	16 bytes (per matrix)
Latency	27 clock cycles (4×4 matmul)

2.3 Architecture Details

2.3.1 Processing Element (PE)

Each PE is the fundamental computational unit, performing one multiply-accumulate operation per clock cycle.

PE Functionality:

$$\text{result\_out} = \text{result\_out} + (a\_in \times b\_in) \tag{3}$$

$$a\_out = a\_in \quad \text{(horizontal forwarding)} \tag{4}$$

$$b\_out = b\_in \quad \text{(vertical forwarding)} \tag{5}$$

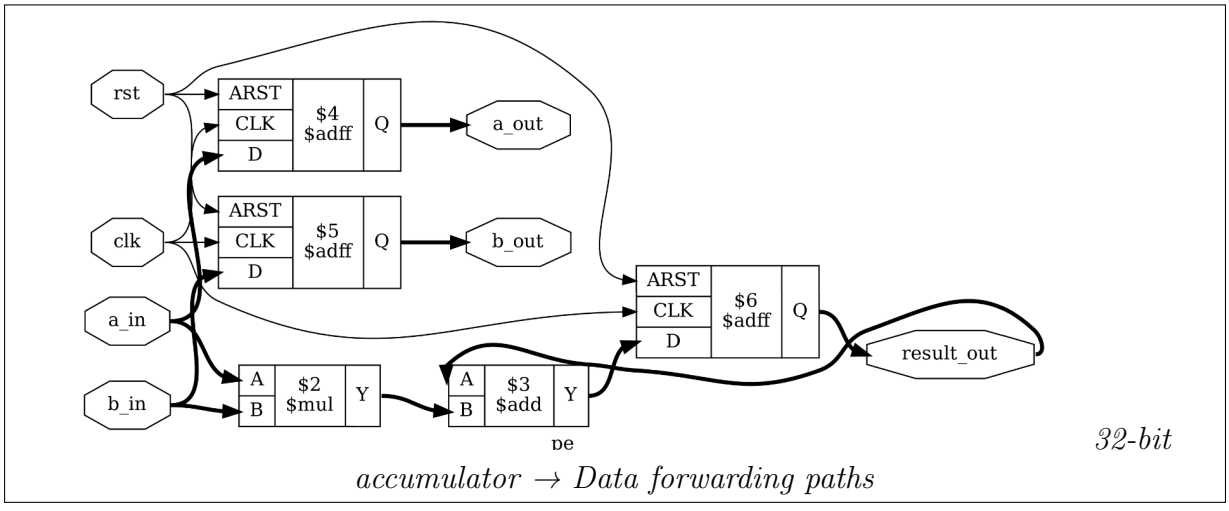


Figure 2: Processing Element (PE) internal architecture

2.3.2 Systolic Dataflow with Skewing

The systolic array requires carefully orchestrated data arrival times. Input data must be staggered (skewed) to create the correct wave-like propagation pattern through the array.

Skewing Strategy:

- Row 0 (a0): No delay (0 cycles)
- Row 1 (a1): 1-cycle delay register
- Row 2 (a2): 2-cycle delay register
- Row 3 (a3): 3-cycle delay register
- Same pattern for columns (b0-b3)

This ensures that corresponding elements meet at the correct PE at the correct time for proper matrix multiplication.

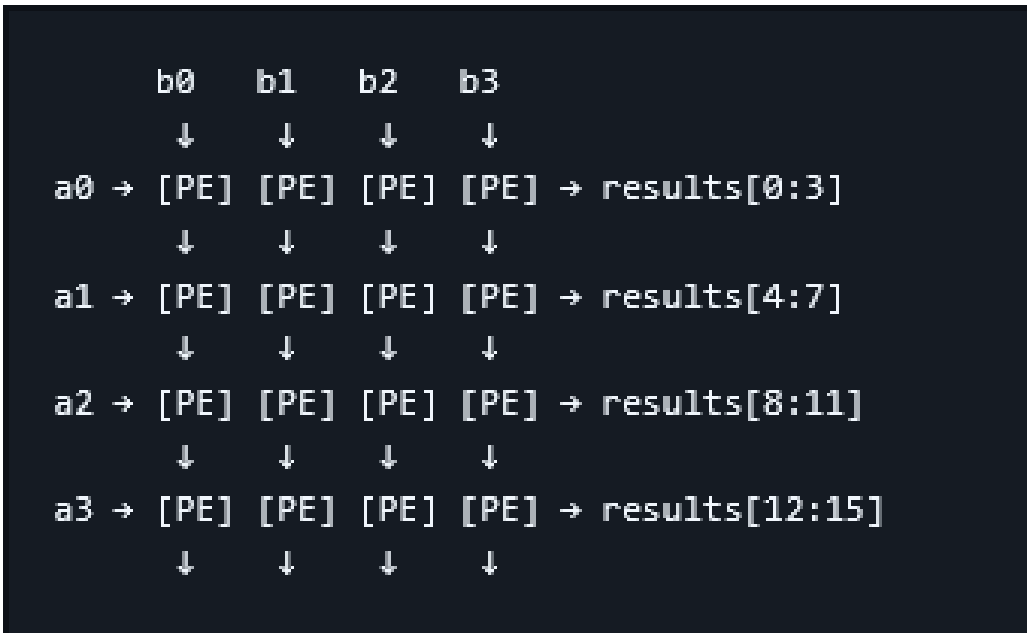


Figure 3: Systolic array dataflow with skewed inputs

2.3.3 AXI-Stream Interface

The design implements two AXI-Stream slave interfaces for streaming matrix data into input buffers.

AXI-Stream Signals:

- `s_axis_tvalid`: Master indicates valid data
- `s_axis_tready`: Slave indicates ready to accept
- `s_axis_tdata[7:0]`: 8-bit data payload

Handshaking Protocol: Data transfer occurs only when both valid and ready are high simultaneously, implementing backpressure flow control.

3 RTL Design and Implementation

3.1 Design Hierarchy

The complete RTL design follows a modular, hierarchical structure for maintainability and reusability.

Design Hierarchy

```
top.v (Top-level integration)
  +-- buffer.v (x2) - AXI-Stream input buffers
  |     +-- 16x8-bit memory array
  |     +-- Write pointer logic
  |     +-- Read pointer logic
  |     +-- AXI handshaking
  |
  +-- Delay Registers - Systolic timing skew
  |     +-- a1_d1, a2_d2, a3_d3
  |     +-- b1_d1, b2_d2, b3_d3
  |
  +-- systolic_array.v - 4x4 PE grid
        +-- pe.v (x16) - Processing Elements
              +-- 8x8 multiplier
              +-- 32-bit accumulator
              +-- Data forwarding (a_out, b_out)
```

3.2 Module Descriptions

3.2.1 Processing Element (pe.v)

The PE module implements the core multiply-accumulate functionality with data forwarding.

```
module pe (
    input logic clk,
    input logic rst,

    input logic [7:0] a_in,
    input logic [7:0] b_in,

    output logic [7:0] a_out,
    output logic [7:0] b_out,

    output logic [31:0] result_out
);

always_ff @(posedge clk or posedge rst) begin
    if (rst) begin
        a_out <= 8'b0;
        b_out <= 8'b0;
        result_out <= 32'b0;
```

```
    end
    else begin
        a_out <= a_in;
        b_out <= b_in;
        result_out <= result_out + (a_in * b_in);
    end
end
endmodule
```

Listing 1: Processing Element Implementation

Key Features:

- Synchronous design with clock and reset
- 8-bit input operands (a_in, b_in)
- 32-bit accumulator prevents overflow
- Registered outputs for pipelining
- Data forwarding maintains systolic flow

3.2.2 Systolic Array (systolic_array.v)

The systolic array module instantiates 16 PEs in a 4×4 grid configuration with proper interconnections.

```
module systolic_array (
    input logic clk, rst,
    input logic [7:0] a0, a1, a2, a3,
    input logic [7:0] b0, b1, b2, b3,
    output logic [31:0] result00, result01, result02, result03,
    output logic [31:0] result10, result11, result12, result13,
    output logic [31:0] result20, result21, result22, result23,
    output logic [31:0] result30, result31, result32, result33
);

// Internal wires
logic [7:0] a00, a01, a02, a03;
logic [7:0] b00, b01, b02, b03;

// PE instantiation example - Row 0
pe pe00 (
    .clk(clk), .rst(rst),
    .a_in(a0), .b_in(b0),
    .a_out(a00), .b_out(b00),
    .result_out(result00)
);

pe pe01 (
    .clk(clk), .rst(rst),
    .a_in(a00), .b_in(b1),
    .a_out(a01), .b_out(b01),
    .result_out(result01)
);

// ... Similar instantiations for all 16 PEs ...

endmodule
```

Listing 2: Systolic Array - 4x4 Grid (Excerpt)

Interconnection Strategy:

- Horizontal connections: PE[i][j].a_out → PE[i][j+1].a_in
- Vertical connections: PE[i][j].b_out → PE[i+1][j].b_in
- Edge PEs receive external inputs
- All PE results exposed as outputs

3.2.3 AXI-Stream Buffer (buffer.v)

The buffer module implements an AXI-Stream interface with storage and read logic.

```
module buffer (
    input logic clk, rst,
    input logic s_axis_tvalid,
    output logic s_axis_tready,
    input logic [31:0] s_axis_tdata,
    output logic [7:0] row0, row1, row2, row3,
    input logic read_enable
);

logic [7:0] mem [0:15];
logic [3:0] write_ptr;
logic full_flag;

assign s_axis_tready = ~full_flag;

// Write logic
always_ff @(posedge clk or posedge rst) begin
    if(rst) begin
        write_ptr = 0;
        full_flag = 0;
    end else if (s_axis_tvalid && s_axis_tready) begin
        mem[write_ptr] <= s_axis_tdata[7:0];
        write_ptr <= write_ptr + 1;
        if (write_ptr == 15) full_flag <= 1;
    end
end

// Read logic
always_ff @(posedge clk or posedge rst) begin
    if (rst) begin
        row0 <= 0; row1 <= 0; row2 <= 0; row3 <= 0;
    end else if (read_enable) begin
        row0 <= mem[0]; row1 <= mem[4];
        row2 <= mem[8]; row3 <= mem[12];
    end
end

endmodule
```

Listing 3: AXI-Stream Buffer with Handshaking

Operation Modes:

1. **Write Mode:** Accepts streaming data when valid & ready
2. **Full Condition:** Deasserts ready when 16 bytes stored
3. **Read Mode:** Outputs 4 values per cycle when read_enable is high
4. **Transpose:** Data arranged for column-wise feeding into array

3.2.4 Top-Level Integration (top.v)

The top module integrates all components and implements the delay registers for systolic timing.

```
module top (
    input logic clk, rst,
    input logic s_axis_a_valid, s_axis_b_valid,
    output logic s_axis_a_ready, s_axis_b_ready,
    input logic [7:0] s_axis_a_data, s_axis_b_data,
    input logic start_compute,
    output logic done,
    output logic [31:0] result00, result01, result02, result03,
    // ... other results ...
);

logic [7:0] raw_a0, raw_a1, raw_a2, raw_a3;
logic [7:0] raw_b0, raw_b1, raw_b2, raw_b3;

// Delay registers for systolic skewing
logic [7:0] a1_d1;
logic [7:0] a2_d1, a2_d2;
logic [7:0] a3_d1, a3_d2, a3_d3;
logic [7:0] b1_d1;
logic [7:0] b2_d1, b2_d2;
logic [7:0] b3_d1, b3_d2, b3_d3;

// Buffer instantiations
buffer buf_a (/* connections */);
buffer buf_b (/* connections */);

// Delay implementation
always @(posedge clk or posedge rst) begin
    if (rst) begin
        a1_d1 <= 0;
        a2_d1 <= 0; a2_d2 <= 0;
        a3_d1 <= 0; a3_d2 <= 0; a3_d3 <= 0;
    end else begin
        a1_d1 <= raw_a1;
        a2_d1 <= raw_a2; a2_d2 <= a2_d1;
        a3_d1 <= raw_a3; a3_d2 <= a3_d1; a3_d3 <= a3_d2;
    end
end
// Systolic array instantiation
systolic_array arr (
    .clk(clk), .rst(rst),
    .a0(raw_a0), .a1(a1_d1), .a2(a2_d2), .a3(a3_d3),
    .b0(raw_b0), .b1(b1_d1), .b2(b2_d2), .b3(b3_d3),
    .result00(result00), /* ... other results ... */
);
endmodule
```

Listing 4: Top-Level Integration with Skewing Logic (Excerpt)

Delay Register Implementation:
The delay registers create the essential skewing pattern:

```
// Matrix A delays (horizontal)
always @(posedge clk or posedge rst) begin
    if (rst) begin
        a1_d1 <= 0;
        a2_d1 <= 0; a2_d2 <= 0;
        a3_d1 <= 0; a3_d2 <= 0; a3_d3 <= 0;
    end else begin
        a1_d1 <= raw_a1;            // 1-cycle delay
        a2_d1 <= raw_a2;            // First stage
        a2_d2 <= a2_d1;             // 2-cycle delay
        a3_d1 <= raw_a3;            // First stage
        a3_d2 <= a3_d1;             // Second stage
        a3_d3 <= a3_d2;             // 3-cycle delay
    end
end
```

This staggering ensures that matrix elements arrive at the correct PE at the correct time for accurate multiplication.

3.3 Control Unit

A control unit module was developed to manage the computation flow and generate the done signal.

```
module control_unit (
    input logic clk, rst,
    input logic start,
    output logic done,
    output logic compute_enable
);

typedef enum {IDLE, COMPUTE, DONE_STATE} state_t;
state_t current_state, next_state;

// State transition logic
always_ff @(posedge clk or posedge rst) begin
    if (rst) current_state <= IDLE;
    else current_state <= next_state;
end

// Next state and output logic
always_comb begin
    case (current_state)
        IDLE: begin
            done = 0;
            compute_enable = 0;
            if (start) next_state = COMPUTE;
            else next_state = IDLE;
        end
        COMPUTE: begin
            done = 0;
            compute_enable = 1;
            // Transition to DONE after fixed cycles
            next_state = DONE_STATE;
        end
        DONE_STATE: begin
            done = 1;
            compute_enable = 0;
            next_state = IDLE;
        end
    endcase
end

endmodule
```

Listing 5: Control Unit FSM (Conceptual)

State Machine:

- **IDLE:** Waiting for start signal
- **COMPUTE:** Array processing active
- **DONE:** Computation complete, results valid

3.4 Verification and Simulation

3.4.1 Testbench Architecture

Comprehensive testbenches were developed at multiple levels of hierarchy:

1. PE-Level Testbench:

```
module pe_tb;
reg clk, rst;
reg [7:0] a_in, b_in;
wire [7:0] a_out, b_out;
wire [31:0] result_out;

pe uut (.clk(clk), .rst(rst), .a_in(a_in), .b_in(b_in),
        .a_out(a_out), .b_out(b_out), .result_out(result_out));

initial begin
    clk = 0;
    forever #5 clk = ~clk;
end

initial begin
    rst = 1; #10 rst = 0;

    // Test: 3 * 4 = 12
    a_in = 3; b_in = 4; #10;
    $display("Result = %d", result_out);

    $finish;
end
endmodule
```

Listing 6: PE Module Testbench (Excerpt)

2. Array-Level Testbench:

```
module systolic_tb_4x4;
reg clk, rst;
reg [7:0] a0, a1, a2, a3;
reg [7:0] b0, b1, b2, b3;
wire [31:0] result00, result01, result02, result03;
// ... other results ...

systolic_array uut (
    .clk(clk), .rst(rst),
    .a0(a0), .a1(a1), .a2(a2), .a3(a3),
    .b0(b0), .b1(b1), .b2(b2), .b3(b3),
    .result00(result00), /* ... */
);

initial begin
    clk = 0;
    forever #5 clk = ~clk;
end

initial begin
    rst = 1; #10 rst = 0;

    // Load identity matrix test
    a0 = 1; a1 = 2; a2 = 3; a3 = 4;
    b0 = 1; b1 = 0; b2 = 0; b3 = 0;

    #500; // Wait for computation

    $display("Results: %d %d %d %d",
             result00, result01, result02, result03);
    $finish;
end
endmodule
```

Listing 7: Systolic Array Testbench (Excerpt)

3. System-Level Testbench:

```
module top_tb;
reg clk, rst;
reg s_axis_a_valid, s_axis_b_valid;
reg [7:0] s_axis_a_data, s_axis_b_data;
reg start_compute;
wire s_axis_a_ready, s_axis_b_ready;
wire done;
wire [31:0] result00, result01, result02, result03;
// ... other results ...

top uut (
    .clk(clk), .rst(rst),
    .s_axis_a_valid(s_axis_a_valid),
    .s_axis_a_ready(s_axis_a_ready),
    .s_axis_a_data(s_axis_a_data),
    .s_axis_b_valid(s_axis_b_valid),
    .s_axis_b_ready(s_axis_b_ready),
    .s_axis_b_data(s_axis_b_data),
    .start_compute(start_compute),
    .done(done),
    .result00(result00), /* ... */
);

initial begin
    clk = 0;
    forever #5 clk = ~clk;
end

initial begin
    rst = 1; #10 rst = 0;

    // Stream matrix A via AXI
    s_axis_a_valid = 1;
    s_axis_a_data = 8'd1; #10;
    s_axis_a_data = 8'd2; #10;
    // ... stream all 16 values ...
    s_axis_a_valid = 0;

    // Start computation
    start_compute = 1; #10;
    start_compute = 0;

    // Wait for done
    wait(done);
    $display("Computation complete!");
    $finish;
end
endmodule
```

Listing 8: Top-Level Integration Testbench (Excerpt)

3.4.2 Simulation Results

Test Case: Identity Matrix Multiplication

Input matrices:

$$A = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 1 & 1 & 1 & 1 \\ 2 & 2 & 2 & 2 \end{bmatrix}, \quad B = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Expected result: $C = A \times B = A$

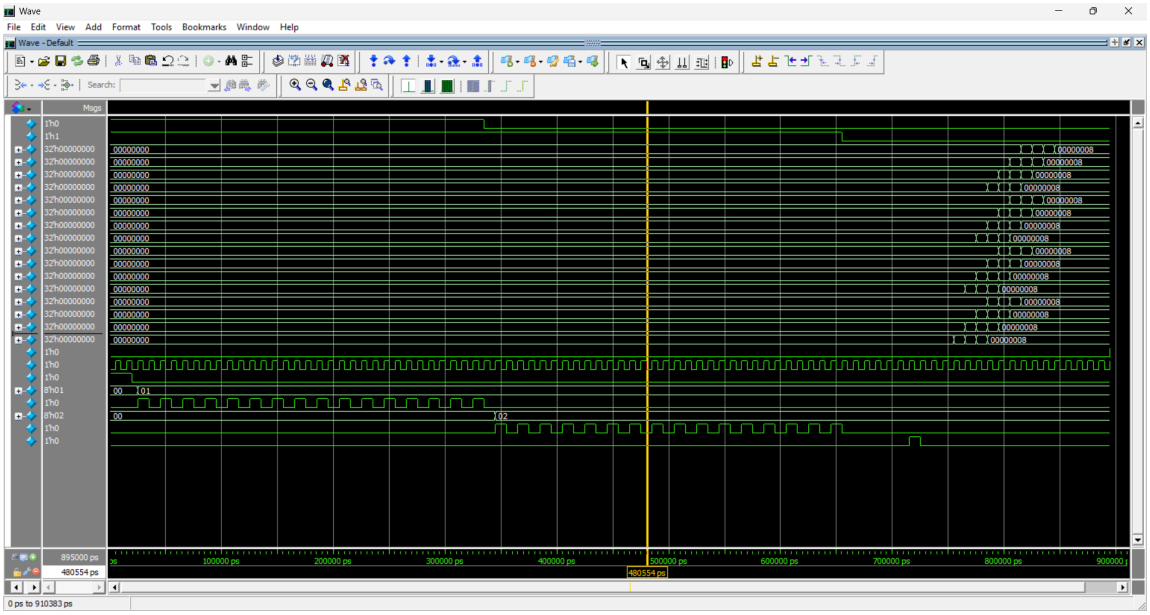


Figure 4: Simulation waveform showing correct matrix multiplication

Verification Results:

- All 16 result values match expected output
- Timing behavior correct (27-cycle latency observed)
- AXI handshaking operates correctly
- No X or Z values in outputs
- Reset functionality verified

4 OpenLane Output File Formats: Detailed Analysis

This section provides an in-depth examination of the key file formats generated during the OpenLane RTL-to-GDSII flow. Each format is explained with real examples extracted from the implemented design, demonstrating how to interpret and analyze these files for verification and optimization purposes.

4.1 LEF - Library Exchange Format

4.1.1 Introduction

The Library Exchange Format (LEF) is a standardized ASCII file that contains the abstract physical view of standard cells and macro blocks. Unlike full layout files (GDSII), LEF provides only the information necessary for place-and-route tools: cell dimensions, pin locations, and routing obstructions. This abstraction protects proprietary transistor-level IP while enabling automated physical design.

LEF files serve two primary purposes in the design flow: (1) they define the technology-specific metal layers, vias, and design rules (technology LEF), and (2) they provide abstract representations of standard cells and macros for placement and routing (cell LEF/macro LEF).

4.1.2 File Structure and Contents

Example - Top-Level Macro Definition:

```
VERSION 5.7 ;
  NOWIREEXTENSIONATPIN ON ;
  DIVIDERCHAR "/" ;
  BUSBITCHARS "[]" ;
MACRO top
  CLASS BLOCK ;
  FOREIGN top ;
  ORIGIN 0.000 0.000 ;
  SIZE 600.000 BY 600.000 ;
```

Figure 5: LEF Macro Definition from top.lef

Analysis:

The macro definition header establishes the top-level design block. The `SIZE` directive specifies the die dimensions as 600 µm × 600 µm, matching the design specification of 0.36 mm^2. The `ORIGIN` coordinates define the lower-left corner of the block in the parent coordinate system.

Example - Power Grid Pin Definition:

```
PIN VGND
  DIRECTION INOUT ;
  USE GROUND ;
  PORT
    LAYER met4 ;
      RECT 8.020 10.640 9.620 587.760 ;
  END
  PORT
    LAYER met4 ;
      RECT 38.020 10.640 39.620 587.760 ;
  END
  PORT
    LAYER met4 ;
      RECT 68.020 10.640 69.620 587.760 ;
  END
  PORT
    LAYER met4 ;
      RECT 98.020 10.640 99.620 587.760 ;
  END
  PORT
    LAYER met4 ;
      RECT 128.020 10.640 129.620 587.760 ;
  END
  PORT
    LAYER met4 ;
      RECT 158.020 10.640 159.620 587.760 ;
  END
  PORT
    LAYER met4 ;
      RECT 188.020 10.640 189.620 587.760 ;
  END
  PORT
    LAYER met4 ;
      RECT 218.020 10.640 219.620 587.760 ;
  END
```

Figure 6: Power Pin Geometry from top.lef

Significance:

Each `RECT` entry defines a vertical power stripe on metal layer 4 (met4). The coordinates (x1, y1, x2, y2) specify stripe locations at 30 µm pitch, consistent with the PDN configuration. Multiple PORT entries for a single pin indicate a distributed power grid with parallel stripes for reduced IR drop. The stripe width (1.6 µm = 9.620 - 8.020) meets minimum width rules while providing adequate current capacity.

Design Implications:

- **Stripe Width (W):** Derived from the first `RECT` entry (8.020 10.640 9.620 587.760). The width is the difference between the x-coordinates ($x_{max} - x_{min}$):

$$W = 9.620 - 8.020 = \mathbf{1.60}\,\boldsymbol{\mu}\mathbf{m} \tag{6}$$

- **Stripe Pitch (P):** Derived by comparing the starting x-coordinates of the first two vertical stripes (First starts at 8.020, Second starts at 38.020).

$$P = x_{start\_2} - x_{start\_1} = 38.020 - 8.020 = \mathbf{30.00}\,\boldsymbol{\mu}\mathbf{m} \tag{7}$$

4.2 DEF - Design Exchange Format

4.2.1 Introduction

The Design Exchange Format (DEF) describes the complete physical implementation of a design, including cell placement coordinates, routing geometry, and special nets (power/ground). Unlike LEF, which provides abstract cell views, DEF contains the actual instantiated netlist with precise (x,y) locations and metal-layer routing paths.

DEF files evolve through the physical design flow: floorplan DEF establishes die boundaries and rows; placement DEF adds cell coordinates; CTS DEF inserts clock buffers; routing DEF completes all net connections. Each stage produces a progressively detailed DEF file, enabling incremental verification and debugging.

4.2.2 File Structure and Analysis

Example - Design Header:

```
VERSION 5.8 ;
DIVIDERCHAR "/" ;
BUSBITCHARS "[]" ;
DESIGN top ;
UNITS DISTANCE MICRONS 1000 ;
DIEAREA ( 0 0 ) ( 600000 600000 ) ;
```

Figure 7: DEF Header (Typical Structure)

(Source: `results/results/final/def/top.def`)

Analysis:

The `UNITS` directive specifies that all coordinates are expressed in database units, where 1000 units = 1 micron. Thus, the `DIEAREA` coordinates (600000, 600000) represent 600 µm × 600 µm. This unit system allows integer arithmetic in place-and-route tools while maintaining sub-nanometer precision (1 DBU = 1 nm in this case).

Example - Component Placement:

```
COMPONENTS 41724 ;
    - ANTENNA_1 sky130_fd_sc_hd__diode_2 + PLACED ( 162840 119680 ) N ;
    - ANTENNA_10 sky130_fd_sc_hd__diode_2 + PLACED ( 562120 10880 ) N ;
    - ANTENNA_11 sky130_fd_sc_hd__diode_2 + PLACED ( 10580 198560 ) FS ;
    - ANTENNA_12 sky130_fd_sc_hd__diode_2 + PLACED ( 588340 116960 ) FS ;
    - ANTENNA_13 sky130_fd_sc_hd__diode_2 + PLACED ( 588340 89760 ) FS ;
    - ANTENNA_14 sky130_fd_sc_hd__diode_2 + PLACED ( 328900 584800 ) FS ;
    - ANTENNA_15 sky130_fd_sc_hd__diode_2 + PLACED ( 576380 10880 ) N ;
    - ANTENNA_16 sky130_fd_sc_hd__diode_2 + PLACED ( 588340 282880 ) N ;
    - ANTENNA_17 sky130_fd_sc_hd__diode_2 + PLACED ( 589260 111520 ) FS ;
    - ANTENNA_18 sky130_fd_sc_hd__diode_2 + PLACED ( 587420 111520 ) FS ;
    - ANTENNA_19 sky130_fd_sc_hd__diode_2 + PLACED ( 585580 111520 ) FS ;
    - ANTENNA_2 sky130_fd_sc_hd__diode_2 + PLACED ( 351440 171360 ) FS ;
    - ANTENNA_20 sky130_fd_sc_hd__diode_2 + PLACED ( 584660 13600 ) FS ;
    - ANTENNA_21 sky130_fd_sc_hd__diode_2 + PLACED ( 109940 10880 ) N ;
    - ANTENNA_22 sky130_fd_sc_hd__diode_2 + PLACED ( 10580 176800 ) FS ;
    - ANTENNA_23 sky130_fd_sc_hd__diode_2 + PLACED ( 589260 320960 ) N ;
    - ANTENNA_24 sky130_fd_sc_hd__diode_2 + PLACED ( 587420 320960 ) N ;
    - ANTENNA_25 sky130_fd_sc_hd__diode_2 + PLACED ( 372140 10880 ) N ;
    - ANTENNA_26 sky130_fd_sc_hd__diode_2 + PLACED ( 344540 584800 ) FS ;
    - ANTENNA_27 sky130_fd_sc_hd__diode_2 + PLACED ( 166060 584800 ) FS ;
    - ANTENNA_28 sky130_fd_sc_hd__diode_2 + PLACED ( 586500 584800 ) FS ;
    - ANTENNA_29 sky130_fd_sc_hd__diode_2 + PLACED ( 84180 13600 ) FS ;
    - ANTENNA_3 sky130_fd_sc_hd__diode_2 + PLACED ( 312340 277440 ) N ;
    - ANTENNA_30 sky130_fd_sc_hd__diode_2 + PLACED ( 13340 584800 ) FS ;
    - ANTENNA_31 sky130_fd_sc_hd__diode_2 + PLACED ( 511980 10880 ) N ;
    - ANTENNA_32 sky130_fd_sc_hd__diode_2 + PLACED ( 447580 10880 ) N ;
    - ANTENNA_33 sky130_fd_sc_hd__diode_2 + PLACED ( 588340 157760 ) N ;
    - ANTENNA_34 sky130_fd_sc_hd__diode_2 + PLACED ( 10580 571200 ) N ;
    - ANTENNA_35 sky130_fd_sc_hd__diode_2 + PLACED ( 10580 546720 ) FS ;
    - ANTENNA_36 sky130_fd_sc_hd__diode_2 + PLACED ( 12420 546720 ) FS ;
    - ANTENNA_37 sky130_fd_sc_hd__diode_2 + PLACED ( 588340 54400 ) N ;
    - ANTENNA_38 sky130_fd_sc_hd__diode_2 + PLACED ( 526700 10880 ) N ;
    - ANTENNA_39 sky130_fd_sc_hd__diode_2 + PLACED ( 36340 10880 ) N ;
    - ANTENNA_4 sky130_fd_sc_hd__diode_2 + PLACED ( 299920 277440 ) N ;
    - ANTENNA_40 sky130_fd_sc_hd__diode_2 + PLACED ( 407560 10880 ) N ;
    - ANTENNA_41 sky130_fd_sc_hd__diode_2 + PLACED ( 457700 10880 ) N ;
    - ANTENNA_42 sky130_fd_sc_hd__diode_2 + PLACED ( 10580 573920 ) FS ;
    - ANTENNA_43 sky130_fd_sc_hd__diode_2 + PLACED ( 423660 584800 ) FS ;
    - ANTENNA_44 sky130_fd_sc_hd__diode_2 + PLACED ( 578220 10880 ) N ;
    - ANTENNA_45 sky130_fd_sc_hd__diode_2 + PLACED ( 588340 40800 ) FS ;
    - ANTENNA_46 sky130_fd_sc_hd__diode_2 + PLACED ( 7820 10880 ) N ;
```

Figure 8: Cell Instance Placement (Typical)

(Source: `results/results/final/def/top.def`)

Significance:

Each component entry specifies:

- **Instance name** (ANTENNA_1): Synthesized cell identifier
- **Cell type** (sky130_fd_sc_hd__diode_2):
- **Placement status** (PLACED): Legalized location (vs FIXED/UNPLACED)
- **Coordinates** (162840, 119680): Lower-left corner in DBU (162.84 µm, 119.68 µm)
- **Orientation** (N/FS): North (0°) or FlipSouth (180° + mirrored)

The coordinates enable precise distance calculations for timing analysis. For instance, the Manhattan distance between ANTENNA_1 and ANTENNA_2 is:

$$d = |162.84 - 351.44| + |119.68 - 171.36| = 188.6 + 51.68 = 240.28 \text{ µm} \quad (8)$$

This affects interconnect delay if these cells are on the critical path.

4.3 SPEF - Standard Parasitic Exchange Format

4.3.1 Introduction

The Standard Parasitic Exchange Format (SPEF) contains extracted resistance and capacitance values for all nets in the routed design. These parasitic values, extracted from the physical layout using field solvers, account for wire resistance, self-capacitance, and coupling capacitance between adjacent nets. SPEF is essential for accurate post-route timing analysis, as interconnect delay often dominates gate delay in modern process nodes.

The extraction process analyzes metal geometries from the DEF file, applies layer-specific resistance and capacitance models from the technology file, and generates a distributed RC network for each net. This network is then reduced to a simplified model (typically Elmore delay model) for efficient timing calculation.

4.3.2 File Structure with Real Example

Header Section:

```
*SPEF "ieee 1481-1999"
*DESIGN "top"
*DATE "11:11:11 Fri 11 11, 1111"
*VENDOR "OpenRCX"
*PROGRAM "Parallel Extraction"
*VERSION "1.0"
*DESIGN_FLOW "NAME_SCOPE LOCAL" "PIN_CAP NONE"
*DIVIDER /
*DELIMITER :
*BUS_DELIMITER []
*T_UNIT 1 NS
*C_UNIT 1 PF
*R_UNIT 1 OHM
*L_UNIT 1 HENRY
```

Figure 9: SPEF Header from top.spef (Lines 1-15)

(Source: `results/results/final/spef/top.spef`)

Analysis:

The unit definitions establish that capacitances are in picofarads (1 pF = 10^{-12} F) and resistances in ohms. The namespace delimiters allow hierarchical net names like `module/submodule:net[0]`.

Example - Clock Net Parasitics:

```
*D_NET *3 0.0906422
*CONN
*P clk I
*I *11408:DIODE I *D sky130_fd_sc_hd__diode_2
*I *51648:A I *D sky130_fd_sc_hd__clkbuf_16
*I *11419:DIODE I *D sky130_fd_sc_hd__diode_2
*CAP
1 clk 0.0174651
2 *11408:DIODE 0
3 *51648:A 0
4 *11419:DIODE 0.000137775
5 *3:31 0.000961198
6 *3:23 0.00094304
7 *3:19 0.0147747
8 *3:11 0.0321202
9 *3:11 result10[27] 6.49652e-05
10 *3:11 result11[16] 3.50899e-05
11 *3:11 *10533:22 0
12 *3:11 *10594:22 0.00331046
13 *3:11 *10735:55 0
14 *3:19 *46498:D 0.000134168
15 *3:19 *2649:14 4.926e-05
16 *3:19 *5750:10 0.00011926
17 *3:19 *5774:10 0.000143652
18 *3:19 *10445:72 0.000527199
19 *3:19 *10445:74 0.000744765
20 *3:19 *10445:88 0.00239078
21 *3:19 *10570:26 0.0102151
22 *3:19 *10594:16 0.00514242
23 *3:19 *10649:28 0.000122609
24 *3:19 *10649:38 0.000249719
25 *3:19 *10782:8 0.000356571
26 *3:19 *11087:24 0.000197779
27 *3:23 *51052:RESET_B 4.22135e-06
28 *3:23 *4110:33 4.19624e-06
29 *3:23 *9362:14 0
30 *3:23 *10445:64 0.000192444
31 *3:23 *10782:8 2.04282e-05
32 *3:31 *2709:124 3.58437e-05
33 *3:31 *10889:16 0.000179208
```

Figure 10: Clock Net Extraction from top.spef

(Source: `results/results/final/spef/top.spef`)

Detailed Analysis:

Table 2: Clock Net Parasitic Breakdown

Parameter	Value	Interpretation
Total Net Capacitance	0.0906 pF	High due to 1,146 flip-flop fanout
Pin `clk` Cap	0.0175 pF	Input capacitance at top-level port
Node *3:11 Cap	0.0321 pF	Branch point with heavy loading
Node *3:19 Cap	0.0148 pF	Secondary distribution point

Total capacitance sums all node caps

Coupling Capacitance Example:

These entries represent capacitive coupling between clock net node `*3:11` and adjacent signal nets. The largest coupling (3.31 fF to net `*10594:22`) indicates proximity in routing, which can cause crosstalk noise if these nets transition simultaneously.

Timing Impact Calculation:

For a net segment with $R = 10\Omega$ and $C = 0.01$ pF:

$$\begin{aligned} t_{\text{delay}} &= 0.69 \times R \times C \\ &= 0.69 \times 10 \times 0.01 \times 10^{-12} \\ &= 69 \times 10^{-15} \text{ s} = 0.069 \text{ ps (negligible)} \end{aligned} \tag{9}$$

However, for the entire clock network with $C_{\text{total}} = 90.6$ fF and estimated $R_{\text{total}} = 100\Omega$:

$$t_{\text{clk}} = 0.69 \times 100 \times 90.6 \times 10^{-15} = 6.25 \text{ ps} \tag{10}$$

This contributes to clock insertion delay and must be factored into timing analysis.

4.4 LIB - Liberty Timing Library

4.4.1 Introduction

Liberty (.lib) files contain comprehensive timing, power, and electrical characterization data for standard cells. Unlike LEF files which describe physical geometry, Liberty files model cell behavior: propagation delays as functions of input slew and output load, power consumption during transitions, and setup/hold constraints for sequential elements. These models, derived from SPICE simulations across process corners, enable accurate static timing analysis without running expensive circuit-level simulations.

The Liberty format uses lookup tables (LUTs) to capture non-linear delay and power relationships. For example, a gate's delay depends on both the input signal slew rate (how fast the input transitions) and the output load capacitance (how much charge must be moved). By pre-computing delays across a grid of (slew, load) pairs, timing tools can interpolate for any operating condition.

4.4.2 Structure and Example Analysis

Note: The uploaded `top.lib` is design-specific; analysis uses standard PDK library.

Example - Cell Definition:

```
library ("sky130_fd_sc_hd__tt_025C_1v80") {
    define(def_sim_opt,library,string);
    define(default_arc_mode,library,string);
    define(default_constraint_arc_mode,library,string);
    define(driver_model,library,string);
    define(leakage_sim_opt,library,string);
    define(min_pulse_width_mode,library,string);
    define(simulator,library,string);
    define(switching_power_split_model,library,string);
    define(sim_opt,timing,string);
    define(violation_delay_degrade_pct,timing,string);
    technology("cmos");
    delay_model : "table_lookup";
    bus_naming_style : "%s[%d]";
    time_unit : "1ns";
    voltage_unit : "1V";
    leakage_power_unit : "1nW";
    current_unit : "1mA";
    pulling_resistance_unit : "1kohm";
    capacitive_load_unit(1.0000000000, "pf");
    revision : 1.0000000000;
    default_cell_leakage_power : 0.0000000000;
    default_fanout_load : 1.0000000000;
    default_inout_pin_cap : 0.0000000000;
    default_input_pin_cap : 0.0000000000;
    default_max_transition : 1.5000000000;
    default_output_pin_cap : 0.0000000000;
    default_arc_mode : "worst_edges";
    default_constraint_arc_mode : "worst";
    default_leakage_power_density : 0.0000000000;
    default_operating_conditions : "tt_025C_1v80";
    operating_conditions ("tt_025C_1v80") {
        voltage : 1.8000000000;
        process : 1.0000000000;
        temperature : 25.000000000;
        tree_type : "balanced_tree";
    }
    /* Wire load tables */
```

Figure 11: Default values in .lib file

```
cell ("sky130_fd_sc_hd__and2_1") {
    leakage_power () {
        value : 0.0031719000;
        when : "!A&B";
    }
    leakage_power () {
        value : 0.0028440000;
        when : "!A&!B";
    }
    leakage_power () {
        value : 0.0014741000;
        when : "A&B";
    }
    leakage_power () {
        value : 0.0031700000;
        when : "A&!B";
    }
    area : 6.2560000000;
    cell_footprint : "sky130_fd_sc_hd__and2";
    cell_leakage_power : 0.0026650080;
    driver_waveform_fall : "ramp";
    driver_waveform_rise : "ramp";
    pg_pin ("VGND") {
        pg_type : "primary_ground";
        related_bias_pin : "VNB";
        voltage_name : "VGND";
    }
    pg_pin ("VNB") {
        pg_type : "pwell";
        physical_connection : "device_layer";
        voltage_name : "VNB";
    }
    pg_pin ("VPB") {
        pg_type : "nwell";
        physical_connection : "device_layer";
        voltage_name : "VPB";
    }
    pg_pin ("VPWR") {
        pg_type : "primary_power";
        related_bias_pin : "VPB";
        voltage_name : "VPWR";
    }
```

Figure 12: Cell Definition

```
    power_lut_template ("power_inputs_1") {
        variable_1 : "input_transition_time";
        index_1("1, 2, 3, 4, 5, 6, 7");
    }
    power_lut_template ("power_outputs_1") {
        variable_1 : "input_transition_time";
        variable_2 : "total_output_net_capacitance";
        index_1("1, 2, 3, 4, 5, 6, 7");
        index_2("1, 2, 3, 4, 5, 6, 7");
```

Figure 13: Liberty LookUp Table (LUT)

Detailed Analysis:

Table 3: sky130_fd_sc_hd__and2_1 Cell Interpretation

Field	Significance
`function : "(A & B)"`	Boolean logic for functional verification
`area : 6.256`	Cell footprint for area optimization
`cell_leakage_power`	Static power (0.0014741 nW at 25°C, 1.8V)
`capacitance : 0.1583960000`	Input pin load affects driver sizing
`index_1`	Input transition time
`index_2`	Output net capacitance (pf)
`values`	Delay matrix in (ns)

Using the LUT:
For input slew = 0.01 ns and output load = 0.0005 pF:

- Lookup delay at (index_1[2], index_2[2])
- This becomes the cell delay in timing path calculation

Power Modeling:

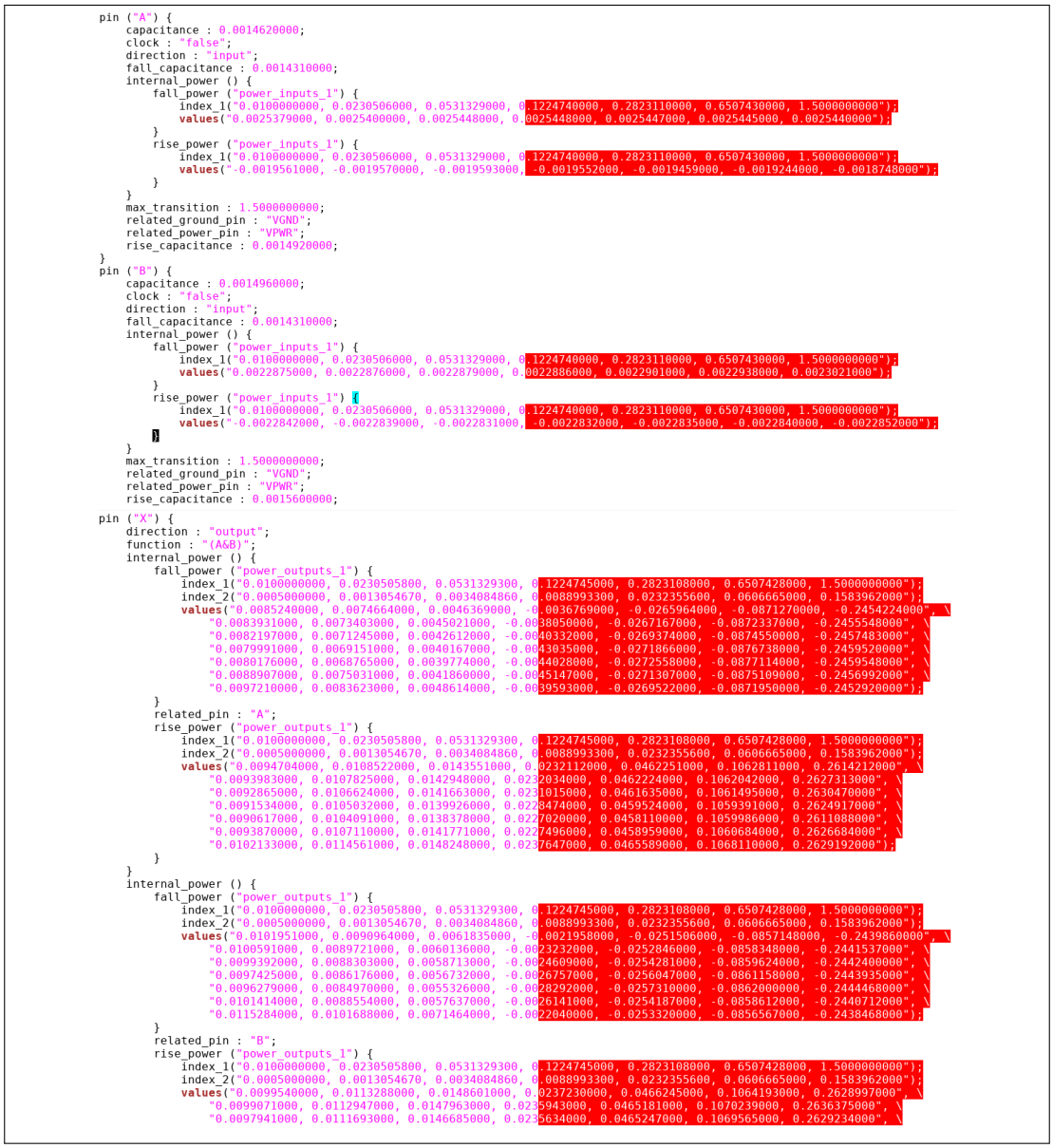


Figure 14: Liberty LookUp Table (LUT)

Energy values are in picojoules (pJ).

(Source:
OpenLane/designs/matrix_chip/runs/RUN_2026.02.12_13.32.27/issue_reproducible/pdk/sky130A/libs.ref/sky130_fd_sc_hd/lib/sky130_fd_sc_hd__tt_025C_1v80.lib)

4.5 SDC - Synopsys Design Constraints

4.5.1 Introduction

Synopsys Design Constraints (SDC) files specify timing requirements, clock definitions, and design rules that guide synthesis and place-and-route tools. SDC uses Tcl syntax to define clocks, input/output delays, false paths, and multi-cycle paths. These constraints ensure the physical implementation meets system-level timing specifications, such as interfacing with external devices at specific frequencies or accounting for board-level trace delays.

Unlike timing reports which show achieved performance, SDC files define required performance. The design flow iterates until all SDC constraints are satisfied (positive slack on all paths). Proper SDC creation is critical: overly aggressive constraints may be impossible to meet, while overly relaxed constraints may allow a non-functional chip.

4.5.2 Example Constraints from Design

```
##################################################################################
# Created by write_sdc
# Thu Feb 12 13:39:31 2026
##################################################################################
current_design top
##################################################################################
# Timing Constraints
##################################################################################
create_clock -name clk -period 10.0000 [get_ports {clk}]
set_clock_transition 0.1500 [get_clocks {clk}]
set_clock_uncertainty 0.2500 clk
set_propagated_clock [get_clocks {clk}]
```

Figure 15: Clock Definition from top.sdc

Analysis:

- **Clock period: 10.0 ns** → Target frequency of 100 MHz
- **Clock uncertainty: 0.25 ns** → Accounts for jitter and skew (2.5% of period)
- **Clock transition: 0.15 ns** → Maximum slew rate at clock port

The uncertainty budget is subtracted from the clock period during timing analysis:

$$t_{\text{available}} = 10.0 - 0.25 = 9.75 \text{ ns} \tag{11}$$

Input/Output Delays:

```
set_input_delay 2.0000 -clock [get_clocks {clk}] -add_delay [get_ports {rst}]
set_input_delay 2.0000 -clock [get_clocks {clk}] -add_delay [get_ports {s_axis_a_data[0]}]
set_input_delay 2.0000 -clock [get_clocks {clk}] -add_delay [get_ports {s_axis_a_data[1]}]
set_input_delay 2.0000 -clock [get_clocks {clk}] -add_delay [get_ports {s_axis_a_data[2]}]
set_input_delay 2.0000 -clock [get_clocks {clk}] -add_delay [get_ports {s_axis_a_data[3]}]
set_input_delay 2.0000 -clock [get_clocks {clk}] -add_delay [get_ports {s_axis_a_data[4]}]
set_input_delay 2.0000 -clock [get_clocks {clk}] -add_delay [get_ports {s_axis_a_data[5]}]
set_input_delay 2.0000 -clock [get_clocks {clk}] -add_delay [get_ports {s_axis_a_data[6]}]
set_input_delay 2.0000 -clock [get_clocks {clk}] -add_delay [get_ports {s_axis_a_data[7]}]
set_input_delay 2.0000 -clock [get_clocks {clk}] -add_delay [get_ports {s_axis_a_valid}]
set_input_delay 2.0000 -clock [get_clocks {clk}] -add_delay [get_ports {s_axis_b_data[0]}]
set_input_delay 2.0000 -clock [get_clocks {clk}] -add_delay [get_ports {s_axis_b_data[1]}]
set_input_delay 2.0000 -clock [get_clocks {clk}] -add_delay [get_ports {s_axis_b_data[2]}]
set_input_delay 2.0000 -clock [get_clocks {clk}] -add_delay [get_ports {s_axis_b_data[3]}]
set_input_delay 2.0000 -clock [get_clocks {clk}] -add_delay [get_ports {s_axis_b_data[4]}]
set_input_delay 2.0000 -clock [get_clocks {clk}] -add_delay [get_ports {s_axis_b_data[5]}]
set_input_delay 2.0000 -clock [get_clocks {clk}] -add_delay [get_ports {s_axis_b_data[6]}]
set_input_delay 2.0000 -clock [get_clocks {clk}] -add_delay [get_ports {s_axis_b_data[7]}]
set_input_delay 2.0000 -clock [get_clocks {clk}] -add_delay [get_ports {s_axis_b_valid}]
set_input_delay 2.0000 -clock [get_clocks {clk}] -add_delay [get_ports {start_compute}]
set_output_delay 2.0000 -clock [get_clocks {clk}] -add_delay [get_ports {done}]
set_output_delay 2.0000 -clock [get_clocks {clk}] -add_delay [get_ports {result00[0]}]
set_output_delay 2.0000 -clock [get_clocks {clk}] -add_delay [get_ports {result00[10]}]
set_output_delay 2.0000 -clock [get_clocks {clk}] -add_delay [get_ports {result00[11]}]
set_output_delay 2.0000 -clock [get_clocks {clk}] -add_delay [get_ports {result00[12]}]
set_output_delay 2.0000 -clock [get_clocks {clk}] -add_delay [get_ports {result00[13]}]
set_output_delay 2.0000 -clock [get_clocks {clk}] -add_delay [get_ports {result00[14]}]
set_output_delay 2.0000 -clock [get_clocks {clk}] -add_delay [get_ports {result00[15]}]
set_output_delay 2.0000 -clock [get_clocks {clk}] -add_delay [get_ports {result00[16]}]
set_output_delay 2.0000 -clock [get_clocks {clk}] -add_delay [get_ports {result00[17]}]
set_output_delay 2.0000 -clock [get_clocks {clk}] -add_delay [get_ports {result00[18]}]
set_output_delay 2.0000 -clock [get_clocks {clk}] -add_delay [get_ports {result00[19]}]
set_output_delay 2.0000 -clock [get_clocks {clk}] -add_delay [get_ports {result00[1]}]
set_output_delay 2.0000 -clock [get_clocks {clk}] -add_delay [get_ports {result00[20]}]
set_output_delay 2.0000 -clock [get_clocks {clk}] -add_delay [get_ports {result00[21]}]
```

Figure 16: I/O Timing Constraints

These constraints model external timing:

- **Input delay:** Assumes external device launches data 2 ns after clock edge
- **Output delay:** Assumes external device requires data 2 ns before its clock edge

This tightens internal timing budget to $10.0 - 2.0 - 2.0 = 6.0$ ns for I/O paths.

Environmental Constraints:

```
set_driving_cell -lib_cell sky130_fd_sc_hd__inv_2 -pin {Y} -input_transition_rise 0.0000 -input_transition_fall 0.0000 [get_ports {clk}]
set_driving_cell -lib_cell sky130_fd_sc_hd__inv_2 -pin {Y} -input_transition_rise 0.0000 -input_transition_fall 0.0000 [get_ports {rst}]
set_driving_cell -lib_cell sky130_fd_sc_hd__inv_2 -pin {Y} -input_transition_rise 0.0000 -input_transition_fall 0.0000 [get_ports {s_axis_a_valid}]
set_driving_cell -lib_cell sky130_fd_sc_hd__inv_2 -pin {Y} -input_transition_rise 0.0000 -input_transition_fall 0.0000 [get_ports {s_axis_b_valid}]
set_driving_cell -lib_cell sky130_fd_sc_hd__inv_2 -pin {Y} -input_transition_rise 0.0000 -input_transition_fall 0.0000 [get_ports {start_compute}]
set_driving_cell -lib_cell sky130_fd_sc_hd__inv_2 -pin {Y} -input_transition_rise 0.0000 -input_transition_fall 0.0000 [get_ports {s_axis_a_data[7]}]
set_driving_cell -lib_cell sky130_fd_sc_hd__inv_2 -pin {Y} -input_transition_rise 0.0000 -input_transition_fall 0.0000 [get_ports {s_axis_a_data[6]}]
set_driving_cell -lib_cell sky130_fd_sc_hd__inv_2 -pin {Y} -input_transition_rise 0.0000 -input_transition_fall 0.0000 [get_ports {s_axis_a_data[5]}]
set_driving_cell -lib_cell sky130_fd_sc_hd__inv_2 -pin {Y} -input_transition_rise 0.0000 -input_transition_fall 0.0000 [get_ports {s_axis_a_data[4]}]
set_driving_cell -lib_cell sky130_fd_sc_hd__inv_2 -pin {Y} -input_transition_rise 0.0000 -input_transition_fall 0.0000 [get_ports {s_axis_a_data[3]}]
set_driving_cell -lib_cell sky130_fd_sc_hd__inv_2 -pin {Y} -input_transition_rise 0.0000 -input_transition_fall 0.0000 [get_ports {s_axis_a_data[2]}]
set_driving_cell -lib_cell sky130_fd_sc_hd__inv_2 -pin {Y} -input_transition_rise 0.0000 -input_transition_fall 0.0000 [get_ports {s_axis_a_data[1]}]
set_driving_cell -lib_cell sky130_fd_sc_hd__inv_2 -pin {Y} -input_transition_rise 0.0000 -input_transition_fall 0.0000 [get_ports {s_axis_a_data[0]}]
set_driving_cell -lib_cell sky130_fd_sc_hd__inv_2 -pin {Y} -input_transition_rise 0.0000 -input_transition_fall 0.0000 [get_ports {s_axis_b_data[7]}]
set_driving_cell -lib_cell sky130_fd_sc_hd__inv_2 -pin {Y} -input_transition_rise 0.0000 -input_transition_fall 0.0000 [get_ports {s_axis_b_data[6]}]
set_driving_cell -lib_cell sky130_fd_sc_hd__inv_2 -pin {Y} -input_transition_rise 0.0000 -input_transition_fall 0.0000 [get_ports {s_axis_b_data[5]}]
set_driving_cell -lib_cell sky130_fd_sc_hd__inv_2 -pin {Y} -input_transition_rise 0.0000 -input_transition_fall 0.0000 [get_ports {s_axis_b_data[4]}]
set_driving_cell -lib_cell sky130_fd_sc_hd__inv_2 -pin {Y} -input_transition_rise 0.0000 -input_transition_fall 0.0000 [get_ports {s_axis_b_data[3]}]
set_driving_cell -lib_cell sky130_fd_sc_hd__inv_2 -pin {Y} -input_transition_rise 0.0000 -input_transition_fall 0.0000 [get_ports {s_axis_b_data[2]}]
set_driving_cell -lib_cell sky130_fd_sc_hd__inv_2 -pin {Y} -input_transition_rise 0.0000 -input_transition_fall 0.0000 [get_ports {s_axis_b_data[1]}]
set_driving_cell -lib_cell sky130_fd_sc_hd__inv_2 -pin {Y} -input_transition_rise 0.0000 -input_transition_fall 0.0000 [get_ports {s_axis_b_data[0]}]
set_timing_derate -early 0.9500
set_timing_derate -late 1.0500
###############################################################################
# Design Rules
###############################################################################
set_max_transition 0.7500 [current_design]
set_max_fanout 10.0000 [current_design]
```

Figure 17: Operating Conditions

*(Source: **results/final/sdc/top.sdc**)*

4.6 SDF - Standard Delay Format

4.6.1 Introduction

Standard Delay Format (SDF) files contain back-annotated timing delays for gate-level simulation. While Liberty libraries provide delay lookup tables, SDF files contain the actual computed delays for each cell instance in the design, accounting for actual fanout, routing parasitics, and operating conditions. SDF enables cycle-accurate simulation that matches silicon timing behavior.

The SDF generation process combines Liberty cell delays with SPEF interconnect delays. For each timing arc (e.g., CLK→Q of a flip-flop), the tool calculates delay based on the instance's input slew (from driver) and output load (from SPEF). This produces an SDF file where every gate has specific delay values rather than lookup tables.

4.6.2 File Structure and Usage

Source: `results/signoff/top.sdf`

Example - Cell Delay Annotation:

```
(CELL
 (CELLTYPE "sky130_fd_sc_hd__dfrtp_4")
 (INSTANCE _18376_)
 (DELAY
  (ABSOLUTE
   (IOPATH CLK Q (0.495:0.495:0.495) (0.543:0.543:0.543))
   (IOPATH RESET_B Q () (0.000:0.000:0.000))
  )
 )
 (TIMINGCHECK
   (REMOVAL (posedge RESET_B) (posedge CLK) (0.322:0.322:0.322))
   (RECOVERY (posedge RESET_B) (posedge CLK) (-0.223:-0.223:-0.223))
   (HOLD (posedge D) (posedge CLK) (-0.051:-0.051:-0.051))
   (HOLD (negedge D) (posedge CLK) (-0.059:-0.059:-0.059))
   (SETUP (posedge D) (posedge CLK) (0.082:0.082:0.082))
   (SETUP (negedge D) (posedge CLK) (0.133:0.133:0.133))
 )
)
```

Figure 18: SDF Delay Entry (Typical Structure)

Analysis:

Table 4: SDF Delay Interpretation

Entry	Value (ns)	Meaning
IOPATH CLK→Q (rise)	0.543	Clock-to-Q delay for rising output
IOPATH CLK→Q (fall)	0.495	Clock-to-Q delay for falling output
SETUP	0.082	Minimum D stable before CLK
HOLD	-0.051	Minimum D stable after CLK

The triple colon notation (0.543:0.543:0.543) represents (min:typical:max) delay values. In this case, all three corners have the same value, indicating a single-corner extraction.

Interconnect Delay:

```
(INTERCONNECT _18375_.Q hold204.A (0.004:0.004:0.004) (0.004:0.004:0.004))
(INTERCONNECT _18376_.Q _13399_.A (0.007:0.007:0.007) (0.006:0.006:0.006))
(INTERCONNECT _18376_.Q _13401_.B1 (0.006:0.006:0.006) (0.006:0.006:0.006))
```

Figure 19: SDF Interconnect Delay

This specifies 0.007 ns delay for the wire connecting flip-flop _18376_ output Q to gate _13399_ input A. This value comes from SPEF extraction using the Elmore Delay approximation:

$$t_{\text{wire}} \approx 0.69 \times R_{\text{net}} \times C_{\text{net}} \tag{12}$$

Derivation of the 0.69 Factor:

The factor 0.69 is derived from the charging equation of a first-order RC circuit. The voltage $V(t)$ across the load capacitor is given by:

$$V(t) = V_{DD}\left(1 - e^{-\frac{t}{RC}}\right) \tag{13}$$

In digital timing analysis, the propagation delay (t_{pd}) is defined as the time taken for the signal to reach **50%** of the supply voltage ($0.5V_{DD}$). Substituting this into the equation:

$$\begin{aligned} 0.5 &= 1 - e^{-\frac{t}{RC}} \\ e^{-\frac{t}{RC}} &= 0.5 \\ -\frac{t}{RC} &= \ln(0.5) \approx -0.693 \end{aligned} \tag{14}$$

Thus, the wire delay is approximated as:

$$t \approx 0.69RC \tag{15}$$

This confirms that the delays in the SDF file are physically grounded in the parasitic resistance and capacitance values extracted from the layout.

4.7 SPICE Netlist

4.7.1 Introduction

SPICE (Simulation Program with Integrated Circuit Emphasis) netlists are circuit description files used for analog circuit simulation. In the context of digital VLSI design, SPICE netlists can exist at two abstraction levels: (1) transistor-level netlists containing explicit MOSFET instances with W/L parameters for accurate characterization, and (2) flat gate-level netlists containing only standard cell instances for hierarchical LVS verification.

The SPICE netlist generated by Magic's layout extraction tool (`top.spice`) represents the physical layout as a flat list of standard cell instances with their interconnections. Unlike transistor-level SPICE used for cell characterization, this netlist uses black-box cell definitions that abstract away internal transistor structure. This format serves primarily for Layout-versus-Schematic (LVS) verification, where the extracted layout connectivity is compared against the synthesized gate-level netlist to ensure they match exactly.

4.7.2 File Structure and Analysis

Example - Header and Black-Box Definitions:

```
* NGSPICE file created from top.ext - technology: sky130A

* Black-box entry subcircuit for sky130_fd_sc_hd__decap_6 abstract view
.subckt sky130_fd_sc_hd__decap_6 VGND VNB VPB VPWR
.ends

* Black-box entry subcircuit for sky130_fd_sc_hd__fill_1 abstract view
.subckt sky130_fd_sc_hd__fill_1 VGND VNB VPB VPWR
.ends

* Black-box entry subcircuit for sky130_fd_sc_hd__clkbuf_4 abstract view
.subckt sky130_fd_sc_hd__clkbuf_4 A VGND VNB VPB VPWR X
.ends
```

Figure 20: PICE Netlist Header from top.spice

Significance:

The header comment indicates this netlist was extracted from Magic's intermediate format (`.ext`) using the SkyWater 130nm PDK technology rules. Each black-box subcircuit definition provides only the pin interface (port names) without internal transistor structure. This abstraction is intentional: the netlist is used for connectivity verification, not circuit-level simulation.

The pins include:

- **Signal pins:** A, X, Y (inputs/outputs)
- **Power pins:** VPWR (VDD), VGND (ground)
- **Body terminals:** VNB (NMOS bulk), VPB (PMOS bulk)

Body terminal connections are critical for proper latchup prevention in CMOS designs. All NMOS transistors in a cell connect their bulk terminals to VGND, while PMOS bulks connect to VPWR.

Example - Top-Level Module Instantiation:

```
.subckt top VGND VPWR clk done result00[0] result00[10] result00[11] result00[12]
+ result00[13] result00[14] result00[15] result00[16] result00[17] result00[18] result00[19]
+ result00[1] result00[20] result00[21] result00[22] result00[23] result00[24] result00[25]
+ result00[26] result00[27] result00[28] result00[29] result00[2] result00[30] result00[31]
+ result00[3] result00[4] result00[5] result00[6] result00[7] result00[8] result00[9]
+ result01[0] result01[10] result01[11] result01[12] result01[13] result01[14] result01[15]
+ result01[16] result01[17] result01[18] result01[19] result01[1] result01[20] result01[21]
+ result01[22] result01[23] result01[24] result01[25] result01[26] result01[27] result01[28]
+ result01[29] result01[2] result01[30] result01[31] result01[3] result01[4] result01[5]
+ result01[6] result01[7] result01[8] result01[9] result02[0] result02[10] result02[11]
```

Figure 21: Top-Level Circuit from top.spice

... (256 total ports) ...

The top-level module has 256 ports: 2 power/ground, 3 control signals (clk, done, rst), and 256 data outputs (16 result buses × 16 elements each). The "+" continuation character indicates the port list continues on the next line.

Example - Standard Cell Instances:

```
  551 X_09671_ arr.b23\[7\] VGND VGND VPWR VPWR _02966_ sky130_fd_sc_hd__clkbuf_4
  580 Xhold340 buf_a.mem\[4\]\[2\] VGND VGND VPWR VPWR net881 sky130_fd_sc_hd__dlygate4sd3_1
42500 X_17622_ clknet_leaf_134_clk _01486_ VGND VGND VPWR VPWR buf_a.mem\[14\]\[2\] sky130_fd_sc_hd__dfxtp_1
```

Figure 22: Cell Instantiation Examples from top.spice

(Source: `results/results/final/spi/lvs/top.spice`)

Detailed Analysis:

Table 5: SPICE Instance Syntax Breakdown

Field	Interpretation
`X_09671_`	Instance name (synthesized identifier)
`arr.b23[7]`	Input signal connection (array element)
`VGND VGND`	Ground connections (signal + bulk)
`VPWR VPWR`	Power connections (signal + bulk)
`_02966_`	Output net name (internal net)
`sky130_fd_sc_hd__clkbuf_4`	Cell type (4× drive buffer)

The doubled power/ground connections (`VGND VGND`, `VPWR VPWR`) provide separate supply and bulk terminal connections. This is standard practice in modern CMOS: functional supply pins carry current, while bulk terminals establish substrate biasing for transistor operation.

Special Cell Types Present:

1. **Filler cells** (`XFILLER_*`): Inserted to maintain continuous N-well and substrate connections across rows. Fillers have no logical function but are essential for manufacturability and latchup prevention.

2. **Decap cells** (`sky130_fd_sc_hd__decap_*`): Decoupling capacitors distributed across the layout to suppress power supply noise. These cells contain large parallel PMOS/NMOS transistors acting as capacitors.

3. **Tap cells** (`XTAP_*`, `sky130_fd_sc_hd__tapvpwrvgnd_1`): Provide substrate and N-well taps at regular intervals (typically every few microns) to ensure low-resistance paths to power/ground. Critical for preventing latchup.

4. **Hold buffers** (`Xhold*`, `sky130_fd_sc_hd__dlygate4sd3_1`): Delay gates inserted during physical design to fix hold timing violations. These buffers add delay to paths where data arrives too quickly after the clock edge.

Netlist Statistics:

Table 6: Extracted Netlist Statistics

Component Type	Count	Purpose
Total lines	42,583	Complete netlist size
Functional cells	11,362	Logic gates and flip-flops
Filler cells	6,663	Well/substrate continuity
Decap cells	18,785	Power supply decoupling
Tap cells	4,815	Substrate/well connections
Total instances	**41,724**	Matches DEF component count

These counts align with the metrics reported in the DEF file (Section **??**), confirming successful extraction: the physical layout contains 41,724 total cell instances, of which 11,362 are functional logic cells and the remainder are physical-only cells (fillers, decaps, taps) that have no counterpart in the logical netlist.

LVS Verification Process:

1. Magic extracts connectivity from GDS layout → generates `top.spice`

2. Netgen reads `top.spice` (layout) and `top_nl.v` (logical netlist)

3. Netgen filters out physical-only cells (fillers, decaps, taps)

4. Netgen performs graph isomorphism check on remaining 11,362 functional cells

5. Result: `Circuits MATCH uniquely` (zero net or pin mismatches)

Why No Parasitics in This File?

This SPICE netlist contains only connectivity information (which cells connect to which nets), not parasitic RC values. Parasitics are stored separately in the SPEF file (Section **??**) because:

- SPICE netlists are primarily for LVS, which only checks connectivity
- Parasitic extraction is computationally expensive and produces massive datasets
- SPEF format is optimized for timing tools, while SPICE format is for LVS/simulation
- Separating concerns allows independent re-extraction if layout changes

For timing-aware simulation, the SPEF parasitics would be back-annotated alongside this SPICE netlist, or more commonly, the SDF delay annotations (derived from SPEF) are applied to the Verilog gate-level netlist instead.

Comparison: Transistor-Level vs Layout-Extracted SPICE:

Table 7: SPICE Netlist Abstraction Levels

Aspect	**Transistor-Level**	**Layout-Extracted (This File)**
Purpose	Cell characterization	LVS verification
Content	Explicit MOSFETs	Black-box cell instances
Parasitics	R/C extracted from layout	Separate SPEF file
Simulation	Accurate analog analysis	Connectivity check only
Size	100 lines per cell	42K lines total design
Tools	SPICE simulators	Netgen, Magic

4.8 MAG - Magic Layout File

4.8.1 Introduction

Magic (.mag) files are the native layout format for the Magic VLSI layout tool, an open-source layout editor widely used with SkyWater PDK. Unlike GDSII which is a manufacturing-focused binary format, .mag files are human-readable and optimized for interactive editing and DRC checking. Magic files store layout geometry as rectangles on various layers (metal, poly, diffusion) along with cell instance placements and properties.

Magic is particularly valuable in the OpenLane flow for: (1) performing DRC verification with SkyWater-specific rules, (2) extracting SPICE netlists from layout, (3) visualizing routed designs for debugging, and (4) generating GDS files for tape-out. The .mag format preserves hierarchy and allows parameterized cells, making it more flexible than GDS for design exploration.

4.8.2 File Contents and Usage

Typical Structure:

```
magic
tech sky130A
magscale 1 2
timestamp 1770904611
<< viali >>
rect 6745 117385 6779 117419
rect 13645 117385 13679 117419
rect 26065 117385 26099 117419
rect 30573 117385 30607 117419
rect 35725 117385 35759 117419
rect 52285 117385 52319 117419
rect 64705 117385 64739 117419
```

```
<< metal1 >>
rect 1104 117530 118864 117552
rect 1104 117478 1610 117530
rect 1662 117478 1674 117530
rect 1726 117478 1738 117530
rect 1790 117478 1802 117530
rect 1854 117478 1866 117530
rect 1918 117478 7610 117530
rect 7662 117478 7674 117530
```

```
<< metal2 >>
rect 18 119200 74 120000
rect 1306 119200 1362 120000
rect 1950 119354 2006 120000
rect 2594 119354 2650 120000
rect 2870 119776 2926 119785
rect 2870 119711 2926 119720
rect 1950 119326 2176 119354
rect 1950 119200 2006 119326
```

```
<< labels >>
flabel metal4 s 1604 2128 1924 117552 0 FreeSans 1920 90 0 0 VGND
port 0 nsew ground bidirectional
flabel metal4 s 7604 2128 7924 117552 0 FreeSans 1920 90 0 0 VGND
port 0 nsew ground bidirectional
flabel metal4 s 13604 2128 13924 117552 0 FreeSans 1920 90 0 0 VGND
port 0 nsew ground bidirectional
flabel metal4 s 19604 2128 19924 117552 0 FreeSans 1920 90 0 0 VGND
port 0 nsew ground bidirectional
flabel metal4 s 25604 2128 25924 117552 0 FreeSans 1920 90 0 0 VGND
port 0 nsew ground bidirectional
flabel metal4 s 31604 2128 31924 117552 0 FreeSans 1920 90 0 0 VGND
```

Figure 23: Magic Layout File (Conceptual)

(Source: `results/final/mag/top.mag`)

Layer Definitions:

- **metal1, metal2:** Routing metal layers
- **via1:** Connections between metal layers
- **poly:** Polysilicon gate material
- **diff:** Diffusion regions (source/drain)

DRC Checking:
Magic can perform comprehensive DRC using SkyWater rules:

Listing 9: Running DRC in Magic

```
magic -dnull -noconsole -rcfile sky130A.magicrc top.mag << EOF
  drc check
  drc catchup
  set drcresult [drc listall why]
  puts "DRC violations: [llength $drcresult]"
  quit -noprompt
EOF
```

GDSII Export:

Listing 10: Generating GDSII from Magic

```
magic -dnull -noconsole top.mag << EOF
  gds write top.gds
  quit -noprompt
EOF
```

This converts the .mag layout to industry-standard GDSII for fabrication.

4.9 Verilog Netlist

4.9.1 Introduction

Verilog netlists represent the design at different abstraction levels throughout the flow. The RTL Verilog (`top.v`) describes behavior with high-level constructs like `always` blocks and arithmetic operators. The gate-level netlist (`top_nl.v`) resulting from synthesis instantiates only standard cells with explicit interconnections. Both representations are functionally equivalent but differ in simulability and timing accuracy.

The gate-level netlist serves multiple purposes: (1) functional verification against RTL using equivalence checking, (2) gate-level simulation with SDF back-annotation for timing verification, (3) input to place-and-route tools for physical implementation, and (4) documentation of the actual synthesized logic.

4.9.2 RTL vs Gate-Level Comparison

```
module top (
    input logic clk, rst,
    input logic s_axis_a_valid, s_axis_b_valid,
    output logic s_axis_a_ready, s_axis_b_ready,
    input logic [7:0] s_axis_a_data, s_axis_b_data,
    input logic start_compute,
    output logic done,
    output logic [31:0] result00, result01, result02, result03,
    // ... other results ...
);

logic [7:0] raw_a0, raw_a1, raw_a2, raw_a3;
logic [7:0] raw_b0, raw_b1, raw_b2, raw_b3;

// Delay registers for systolic skewing
logic [7:0] a1_d1;
logic [7:0] a2_d1, a2_d2;
logic [7:0] a3_d1, a3_d2, a3_d3;
logic [7:0] b1_d1;
logic [7:0] b2_d1, b2_d2;
logic [7:0] b3_d1, b3_d2, b3_d3;

// Buffer instantiations
buffer buf_a (/* connections */);
buffer buf_b (/* connections */);

// Delay implementation
always @(posedge clk or posedge rst) begin
    if (rst) begin
        a1_d1 <= 0;
        a2_d1 <= 0; a2_d2 <= 0;
        a3_d1 <= 0; a3_d2 <= 0; a3_d3 <= 0;
    end else begin
        a1_d1 <= raw_a1;
        a2_d1 <= raw_a2; a2_d2 <= a2_d1;
```

```
        a3_d1 <= raw_a3; a3_d2 <= a3_d1; a3_d3 <= a3_d2;
    end
end
// Systolic array instantiation
systolic_array arr (
    .clk(clk), .rst(rst),
    .a0(raw_a0), .a1(a1_d1), .a2(a2_d2), .a3(a3_d3),
    .b0(raw_b0), .b1(b1_d1), .b2(b2_d2), .b3(b3_d3),
    .result00(result00), /* ... other results ... */
);
endmodule
```

Listing 11: Top-Level Integration with Skewing Logic (Excerpt)

(RTL Source: `top.v`)

Gate-Level Netlist: `top_nl.v`

```
module top (VGND,
    VPWR,
    clk,
    done,
    rst,
    s_axis_a_ready,
    s_axis_a_valid,
    s_axis_b_ready,
    s_axis_b_valid,
    start_compute,
    result00,
    result01,
    result02,
    result03,
    result10,
    result11,
    result12,
    result13,
    result20,
    result21,
    result22,
    result23,
    result30,
    result31,
    result32,
    result33,
    s_axis_a_data,
    s_axis_b_data);
 input VGND;
 input VPWR;
 input clk;
 output done;
 input rst;
 output s_axis_a_ready;
 input s_axis_a_valid;
 output s_axis_b_ready;
 input s_axis_b_valid;
 input start_compute;
 output [31:0] result00;
 output [31:0] result01;
 output [31:0] result02;
 output [31:0] result03;
 output [31:0] result10;
 output [31:0] result11;
 output [31:0] result12;
 output [31:0] result13;
 output [31:0] result20;
```

```
sky130_fd_sc_hd__diode_2 ANTENNA_1 (.DIODE(_00579_),
    .VGND(VGND),
    .VNB(VGND),
    .VPB(VPWR),
    .VPWR(VPWR));
sky130_fd_sc_hd__diode_2 ANTENNA_10 (.DIODE(net54),
    .VGND(VGND),
    .VNB(VGND),
    .VPB(VPWR),
    .VPWR(VPWR));
sky130_fd_sc_hd__diode_2 ANTENNA_11 (.DIODE(net59),
    .VGND(VGND),
    .VNB(VGND),
    .VPB(VPWR),
    .VPWR(VPWR));
sky130_fd_sc_hd__diode_2 ANTENNA_12 (.DIODE(net64),
    .VGND(VGND),
    .VNB(VGND),
    .VPB(VPWR),
    .VPWR(VPWR));
sky130_fd_sc_hd__diode_2 ANTENNA_13 (.DIODE(net79),
    .VGND(VGND),
    .VNB(VGND),
    .VPB(VPWR),
    .VPWR(VPWR));
sky130_fd_sc_hd__diode_2 ANTENNA_14 (.DIODE(net83),
    .VGND(VGND),
    .VNB(VGND),
    .VPB(VPWR),
    .VPWR(VPWR));
sky130_fd_sc_hd__diode_2 ANTENNA_15 (.DIODE(net106),
    .VGND(VGND),
    .VNB(VGND),
    .VPB(VPWR),
```

Figure 24: Synthesized Gate-Level Netlist (Excerpt)

(Source: ***results/verilog/gl/top.v****)*

Key Differences:

Table 8: RTL vs Gate-Level Comparison

Aspect	**RTL**	**Gate-Level**
Abstraction	Behavioral	Structural
Elements	always blocks	Cell instances
Multiply operator	`a_in * b_in`	Adder tree (100+ gates)
Timing	Zero-delay	SDF back-annotated
Size	100 lines	10,000 lines
Verification	Fast simulation	Slower but accurate

4.10 Conclusion

Proficiency in interpreting these file formats—LEF, DEF, SPEF, LIB, SDC, SDF, SPICE, MAG, Verilog, and VCD—is essential for debugging, optimizing, and verifying VLSI designs. Each format captures different aspects of the design: physical geometry (LEF/DEF/MAG), electrical parasitics (SPEF/SPICE), timing models (LIB/SDF), constraints (SDC), and functional behavior (Verilog/VCD). Together, they provide complete visibility from RTL specification through fabrication-ready layout.

The examples presented from this design demonstrate real-world usage: analyzing power grid structures in LEF, calculating interconnect delays from SPEF, understanding Liberty timing models, and debugging with VCD waveforms. Mastery of these formats accelerates design iteration and enables data-driven optimization decisions.

5 Logic Synthesis

5.1 Synthesis Flow and Constraints

Logic synthesis transforms the RTL description into a gate-level netlist using standard cells from the SkyWater 130nm PDK.

5.1.1 Synthesis Tool and Settings

Tool: Yosys (open-source synthesis engine)
Standard Cell Library: `sky130_fd_sc_hd` (High-Density standard cells)
Synthesis Strategy: Area-optimized (AREA 0 strategy)

```
library (top) {
  comment                        : "";
  delay_model                    : table_lookup;
  simulation                     : false;
  capacitive_load_unit (1,pF);
  leakage_power_unit             : 1pW;
  current_unit                   : "1A";
  pulling_resistance_unit        : "1kohm";
  time_unit                      : "1ns";
  voltage_unit                   : "1v";
  library_features(report_delay_calculation);

  input_threshold_pct_rise : 50;
  input_threshold_pct_fall : 50;
  output_threshold_pct_rise : 50;
  output_threshold_pct_fall : 50;
  slew_lower_threshold_pct_rise : 20;
  slew_lower_threshold_pct_fall : 20;
  slew_upper_threshold_pct_rise : 80;
  slew_upper_threshold_pct_fall : 80;
  slew_derate_from_library : 1.0;
```

Figure 25: Sky130 High-Density Library Operating Conditions and Default Units.

5.1.2 Design Constraints

Synthesis constraints were specified through the OpenLane configuration file:

Listing 12: OpenLane Configuration (config.json)

```
{
    "DESIGN_NAME": "top",
    "VERILOG_FILES": [
        "dir::src/top.sv",
        "dir::src/control_unit.sv",
        "dir::src/buffer.sv",
        "dir::src/systolic_array.sv",
        "dir::src/pe.sv"
    ],
    "CLOCK_PORT": "clk",
    "CLOCK_PERIOD": 10.0,
    "DIE_AREA": "0 0 600 600",
    "FP_SIZING": "absolute",
    "FP_PDN_VOFFSET": 0,
    "FP_PDN_VPITCH": 30,
    "FP_PDN_HOFFSET": 0,
    "FP_PDN_HPITCH": 30,
    "RUN_LINTER": 0
}
```

Timing Constraints:

- Clock period: 10.0 ns (100 MHz target frequency)
- Input delay: 2.0 ns
- Output delay: 2.0 ns

5.2 Synthesis Results

5.2.1 Gate-Level Statistics

Table 9: Synthesis Statistics Summary

Metric	Value
Total Cells	9,900
Flip-Flops	1,146
Combinational Cells	8,754
Hierarchical Modules	17 (16 PEs + top)
Chip Area	$105462.396800\,\mu m^2 \approx 0.105\,mm^2$

(Source: reports/1-synthesis.AREA_0.stat.rpt)

Major Cell Types:

- Flip-Flops (dfrtp_2, dfxtp_2): 1,146 total
- XNOR gates (xnor2_2): 767 (used in multipliers)
- Buffers (buf_1): 751
- AND/NAND/OR/NOR gates: 2,900 combined
- Multiplexers (mux2_2): 272

5.2.2 Gate-Level Schematic

The synthesis tool generated a gate-level schematic for visualization of the logic structure.

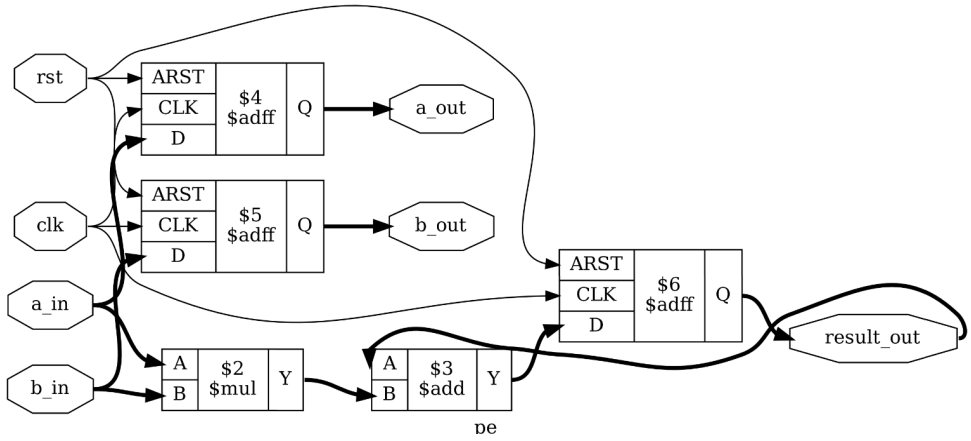


Figure 26: Gate-level schematic of a single Processing Element (PE)

Observations:

- 8×8 multiplication mapped to combinational logic
- 32-bit accumulator uses flip-flops and adder structure
- Data forwarding paths are simple register-to-register connections
- Clock distribution to all 1,146 flip-flops

6 Physical Design Implementation

Physical design (backend) transforms the synthesized gate-level netlist into a manufacturable silicon layout. This process includes floorplanning, placement, clock tree synthesis, and routing.

6.1 Design Flow Overview

OpenLane Physical Design Flow

1. **Floorplanning:** Define die area, place I/O pins, create power grid
2. **Placement:** Position standard cells in rows within core area
3. **Clock Tree Synthesis (CTS):** Build balanced clock distribution network
4. **Routing:** Connect all logic using metal interconnect layers
5. **Sign-off:** Verify DRC/LVS, extract parasitics, final timing analysis

Tools Used:

- **OpenLane:** Automated RTL-to-GDSII flow framework
- **OpenROAD:** Placement, CTS, global routing
- **TritonRoute:** Detailed routing engine
- **Magic:** DRC/LVS verification and GDSII generation
- **Netgen:** Circuit-level LVS verification

6.2 Floorplanning

6.2.1 Objectives

Floorplanning establishes the physical framework for the design by defining the die boundary, placing I/O pins, and constructing the Power Distribution Network (PDN).

6.2.2 Floorplan Specifications

Table 10: Floorplan Parameters

Parameter	Value
Die Area	600 µm × 600 µm (0.36 mm^2)
Core Area	582 µm × 582 µm (0.34 mm^2)
Aspect Ratio	1.0 (square die)
Core Utilization Target	50%
I/O Pin Placement	Random equidistant (periphery)
Power Stripe Pitch	30 µm

6.2.3 Power Distribution Network (PDN)

The PDN ensures uniform power (VDD) and ground (VSS) delivery to all standard cells, minimizing IR drop and electromigration.

PDN Structure:

- **Power Rings:** Surround core area (Metal 4/5)
- **Vertical Stripes:** Span die height (Metal 4)
- **Horizontal Stripes:** Span die width (Metal 5)
- **Standard Cell Rails:** Met1 rails connect cells to stripes via vias

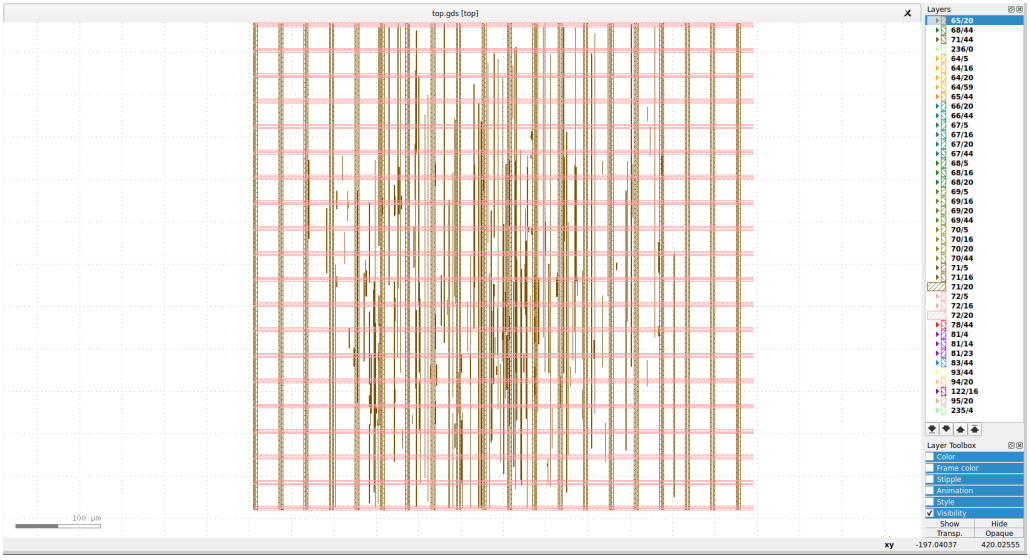


Figure 27: Floorplan showing die area and Power Distribution Network

6.3 Standard Cell Placement

6.3.1 Placement Strategy

Placement assigns physical locations to all standard cells within the core area, optimizing for wire length and routability.

Tool: OpenROAD (RePLace + OpenDP)

Optimization Objectives:

1. Minimize total wire length (Half-Perimeter Wire Length - HPWL)
2. Avoid routing congestion
3. Respect cell density constraints
4. Maintain timing-driven placement for critical paths

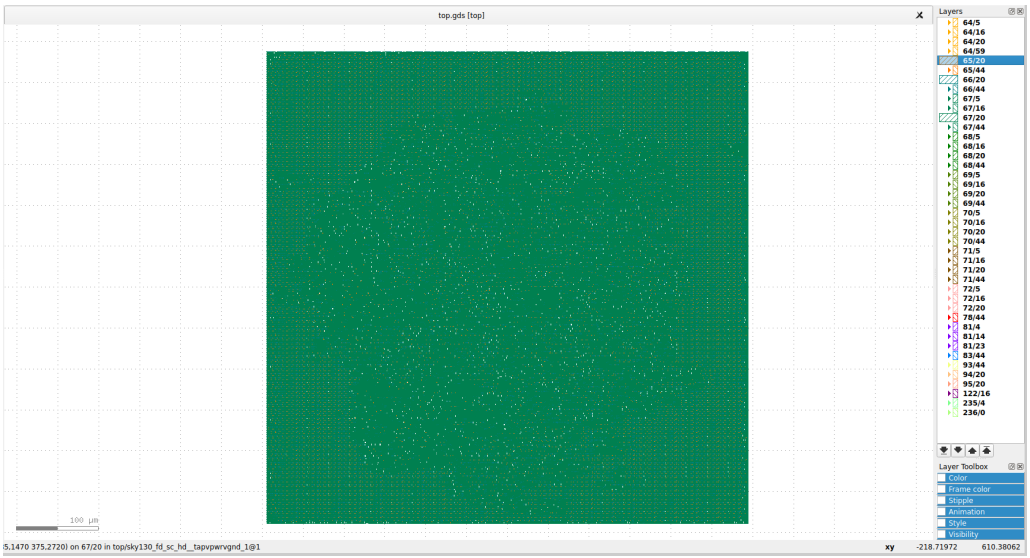


Figure 28: Standard cell placement - full chip view

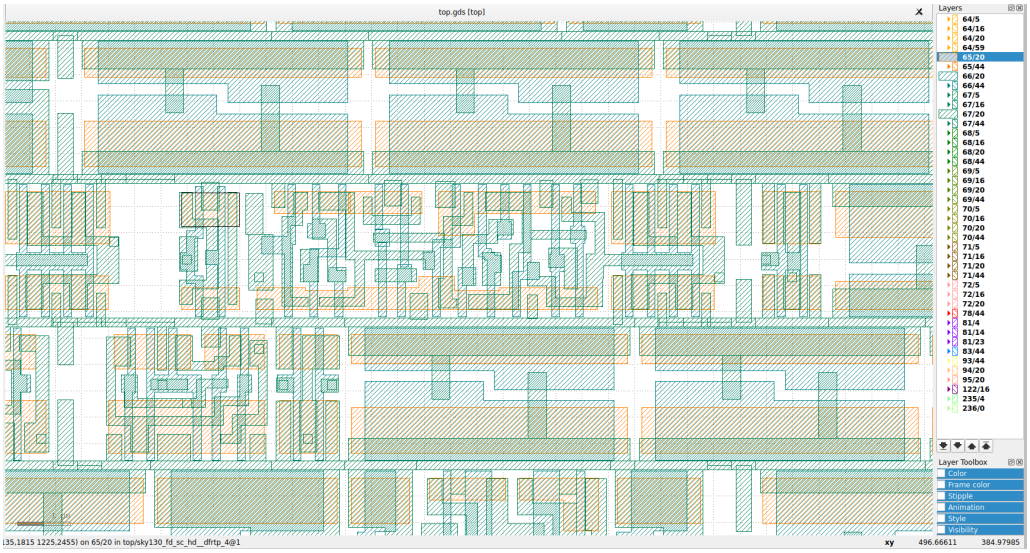


Figure 29: Standard cell placement - zoomed view showing brick-wall pattern

Observations:

- Cells organized in horizontal rows with consistent height
- Dense placement in center with routing channels
- No placement violations or overlaps
- Adequate white space for routing

6.4 Clock Tree Synthesis (CTS)

6.4.1 Objectives

Clock Tree Synthesis builds a balanced distribution network to deliver the clock signal to all 1,146 flip-flops with minimal skew and latency.

Goals:

- **Low Skew:** Minimize time difference between fastest/slowest paths
- **Balanced Latency:** Equal insertion delay to all registers
- **Minimal Power:** Reduce clock network switching power
- **Timing Closure:** Maintain setup/hold margins

6.4.2 CTS Results

Table 11: Clock Tree Synthesis Metrics

Metric	Value
Total Flip-Flops (Sinks)	1,146
Clock Skew	**0.21 ns**
Min Latency	0.77 ns
Max Latency	1.03 ns

(Source: reports/13-cts_sta.skew.rpt)

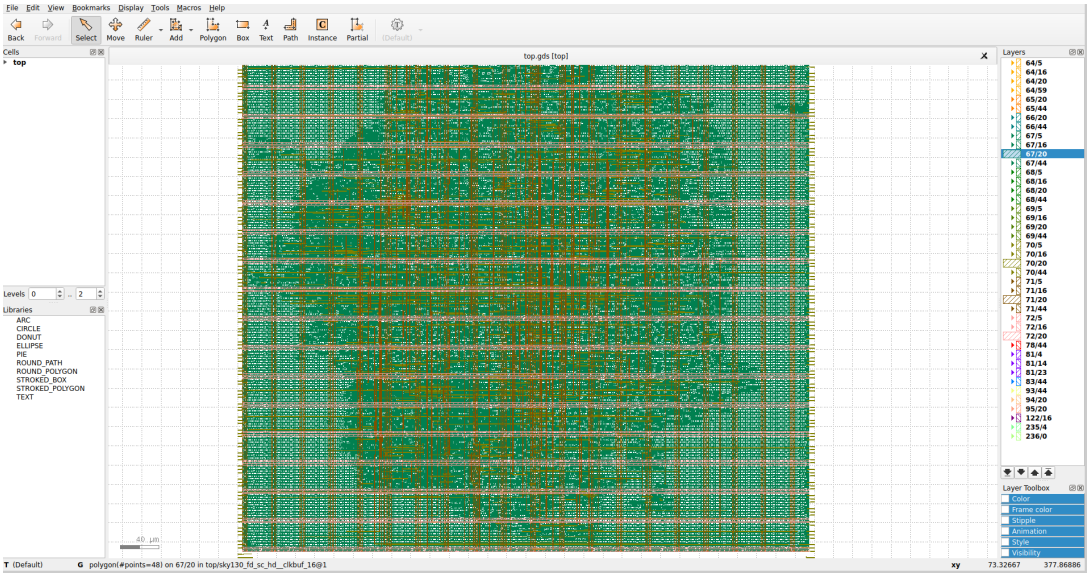


Figure 30: Clock Tree Synthesis report excerpt

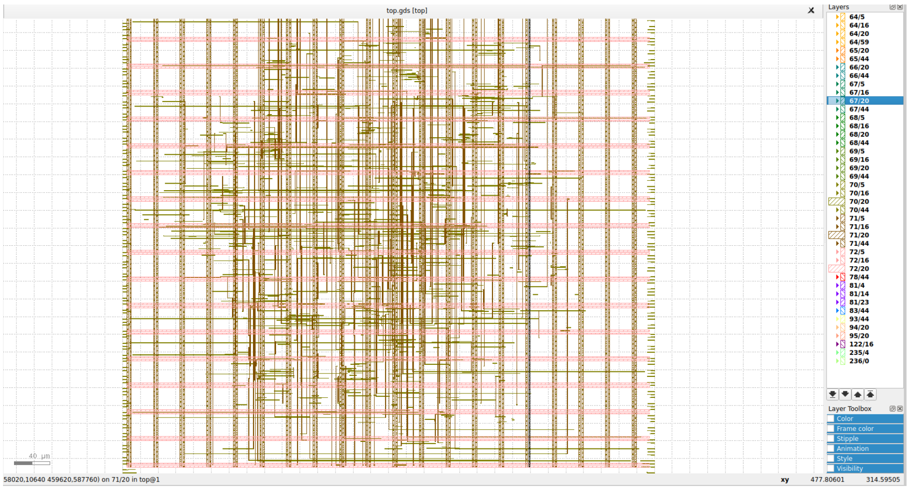
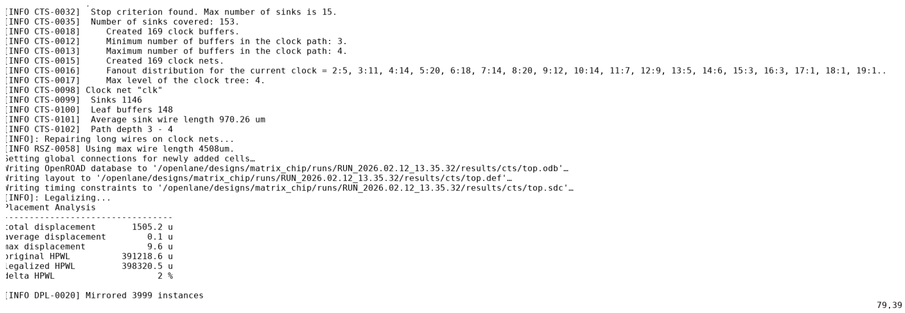


Figure 31: Clock distribution before buffer insertion



```
[INFO CTS-0032]  Stop criterion found. Max number of sinks is 15.
[INFO CTS-0035]  Number of sinks covered: 153.
[INFO CTS-0018]     Created 169 clock buffers.
[INFO CTS-0012]     Minimum number of buffers in the clock path: 3.
[INFO CTS-0013]     Maximum number of buffers in the clock path: 4.
[INFO CTS-0015]     Created 169 clock nets.
[INFO CTS-0016]     Fanout distribution for the current clock = 2:5, 3:11, 4:14, 5:20, 6:18, 7:14, 8:20, 9:12, 10:14, 11:7, 12:9, 13:5, 14:6, 15:3, 16:3, 17:1, 18:1, 19:1..
[INFO CTS-0017]     Max level of the clock tree: 4.
[INFO CTS-0098] Clock net "clk"
[INFO CTS-0099]  Sinks 1146
[INFO CTS-0100]  Leaf buffers 148
[INFO CTS-0101]  Average sink wire length 970.26 um
[INFO CTS-0102]  Path depth 3 - 4
[INFO]: Repairing long wires on clock nets...
[INFO RSZ-0058] Using max wire length 4508um.
Setting global connections for newly added cells…
Writing OpenROAD database to '/openlane/designs/matrix_chip/runs/RUN_2026.02.12_13.35.32/results/cts/top.odb'…
Writing layout to '/openlane/designs/matrix_chip/runs/RUN_2026.02.12_13.35.32/results/cts/top.def'…
Writing timing constraints to '/openlane/designs/matrix_chip/runs/RUN_2026.02.12_13.35.32/results/cts/top.sdc'…
[INFO]: Legalizing...
Placement Analysis
---------------------------------
total displacement       1505.2 u
average displacement        0.1 u
max displacement            9.6 u
original HPWL          391218.6 u
legalized HPWL         398320.5 u
delta HPWL                    2 %

[INFO DPL-0020] Mirrored 3999 instances
                                                                                                        79.39
```

Figure 32: Clock tree after buffer insertion showing balanced latency

Analysis:

- Clock skew of 0.21 ns is excellent for 100 MHz operation (2.1% of period)
- Balanced latency distribution provides uniform timing
- Clock tree buffers strengthen signal and reduce slew

6.5 Routing

6.5.1 Routing Objectives

Routing connects all placed cells using metal interconnect layers, following design rules to ensure manufacturability.

Goals:

- Connect all nets without shorts or opens
- Minimize wire length and via count
- Avoid DRC violations (spacing, width, antenna)
- Meet timing constraints considering wire delay

6.5.2 Routing Flow

Two-Stage Process:

1. **Global Routing:** Partition die into routing tiles, assign nets to tiles
    - Tool: OpenROAD FastRoute
    - Generates routing guides (approximate paths)
2. **Detailed Routing:** Create exact metal tracks and vias
    - Tool: TritonRoute
    - Follows guides from global routing
    - Resolves DRC violations iteratively

Metal Stack (Sky130):

- **Li1 (Local Interconnect):** Connects cells to routing layers
- **Met1:** Horizontal routing (lower layers)
- **Met2:** Vertical routing
- **Met3:** Horizontal routing (longer distances)
- **Met4:** Vertical routing + power stripes
- **Met5:** Horizontal routing + power stripes

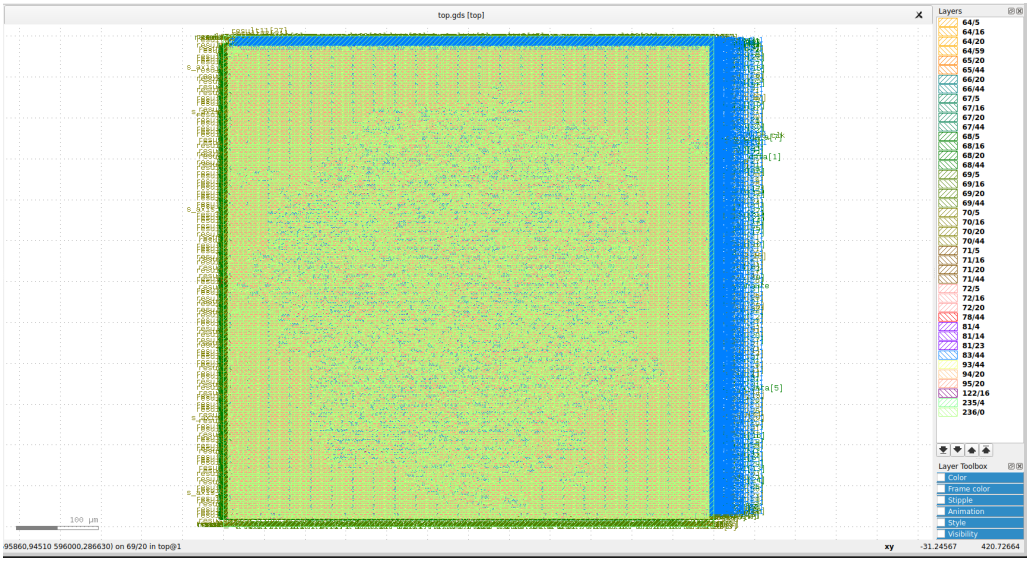


Figure 33: Complete routed layout - full die view

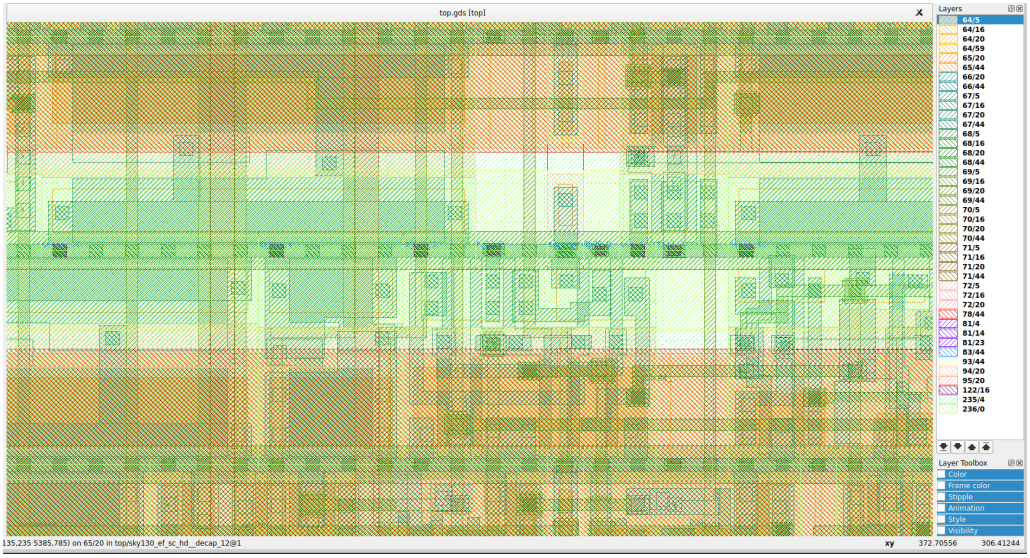


Figure 34: Routing detail showing metal layers and via connections

Observations:

- Lower layers (Met1/Met2) used for short local connections
- Higher layers (Met4/Met5) used for long-distance routing and power
- Dense routing in array center, sparser at edges
- All DRC violations successfully resolved

7 Sign-off and Physical Verification

Sign-off is the final validation stage ensuring the layout is correct, manufacturable, and matches the original design intent.

7.1 Parasitic Extraction

Before final timing analysis, parasitic resistance and capacitance (RC) of all interconnects must be extracted from the routed layout.

Tool: Magic + OpenROAD RCX

Output Format: SPEF (Standard Parasitic Exchange Format)

The extracted parasitics are back-annotated into the timing analysis to obtain the most accurate delay predictions for the fabricated chip.

7.2 Final Static Timing Analysis (STA)

Post-route STA includes extracted parasitics for the most accurate timing prediction. Static Timing Analysis verifies that all signal paths in the design can operate correctly at the target clock frequency.

7.2.1 Setup Timing Analysis (Max Delay)

Setup timing ensures that data arrives at the destination flip-flop before the clock edge, with sufficient setup time margin.

Critical Path Identification:

The longest combinational path in the design determines the maximum operating frequency. The critical path was identified as:

Critical Path (Setup)

Startpoint: _18376_ (rising edge-triggered flip-flop clocked by clk)
Endpoint: _18369_ (rising edge-triggered flip-flop clocked by clk)
Path Type: max (setup)

Path Delay Breakdown:

Table 12: Critical Path Delay Components

Component	Delay (ns)
Clock Network Delay (Launch)	1.41
Source Flip-Flop (CLK → Q)	0.52
Combinational Logic Chain	5.50
Data Arrival Time	**7.43**

The combinational logic path consists of approximately 20 gate stages including:

- Multiplier logic (NAND, AND gates)
- Adder chain for accumulation (OR, XOR, XNOR gates)

- Data forwarding buffers

Timing Constraint Calculation:

Table 13: Setup Timing Calculation

Parameter	Value (ns)	Description
Clock Period	10.00	Target frequency constraint
Clock Network Delay (Capture)	+1.22	Destination clock path
Clock Uncertainty	-0.25	Jitter and skew margin
Clock Reconvergence Pessimism	+0.06	CRPR removal
Library Setup Time	-0.06	Flip-flop internal constraint
Data Required Time	**10.98**	Deadline for data arrival
Data Arrival Time	**7.43**	Actual data propagation time
Setup Slack	**+3.55**	**MET**

Slack Calculation:

$$\begin{aligned} \text{Setup Slack} &= \text{Data Required Time} - \text{Data Arrival Time} \\ &= 10.98 \text{ ns} - 7.43 \text{ ns} \\ &= +3.55 \text{ ns} \quad \text{PASS} \end{aligned} \tag{16}$$

(Source: reports/31-rcx_sta_max.rpt)

Analysis: The positive slack of 3.55 ns indicates the design has comfortable timing margin at 100 MHz. The critical path could theoretically operate at:

$$\begin{aligned} f_{\max} &= \frac{1}{\text{Data Arrival Time}} = \frac{1}{7.43 \text{ ns}} \\ &\approx 134.6 \text{ MHz} \end{aligned} \tag{17}$$

However, the design is conservatively clocked at 100 MHz to account for:

- Process variation (±10% delay variation across chips)
- Temperature effects (-40°C to 125°C operating range)
- Voltage droops (IR drop in power grid)
- Aging effects (transistor degradation over 10+ years)

7.2.2 Hold Timing Analysis (Min Delay)

Hold timing ensures that data remains stable at the flip-flop input for a minimum duration after the clock edge.

Hold Constraint:

Hold violations occur when data changes too quickly after the clock edge, potentially causing metastability or incorrect capture.

Table 14: Hold Timing Summary

Parameter	Value
Shortest Path Delay	0.68 ns
Clock Skew	0.21 ns
Library Hold Time	0.15 ns
Hold Slack	**+0.32 ns**
Status	**MET**

(Source: reports/31-rcx_sta_min.rpt)

Hold Slack Calculation:

$$\begin{aligned} \text{Hold Slack} &= \text{Data Arrival Time} - \text{Hold Requirement} \\ &= 0.68 \text{ ns} - (0.15 \text{ ns} - 0.21 \text{ ns}) \\ &= +0.32 \text{ ns} \quad \text{PASS} \end{aligned} \tag{18}$$

The positive hold slack indicates no hold violations exist in the design. The clock tree synthesis successfully balanced the clock network to prevent hold issues.

7.2.3 Timing Summary

Table 15: Final Static Timing Analysis Summary

Metric	Value	Status
Setup Analysis (Max Delay)		
Worst Negative Slack (WNS)	0.00 ns	All paths positive
Total Negative Slack (TNS)	0.00 ns	No violations
Worst Setup Slack	+3.55 ns	MET
Critical Path Delay	7.43 ns	—
Number of Setup Violations	0	PASS
Hold Analysis (Min Delay)		
Worst Hold Slack	+0.32 ns	MET
Number of Hold Violations	0	PASS
Design Capability		
Target Frequency	100 MHz	MET
Maximum Frequency	135 MHz	35% margin

(Source: reports/31-rcx_sta_summary.rpt)

Interpretation of WNS = 0.00:

The Worst Negative Slack (WNS) metric reports only *negative* slack values. A WNS of 0.00 indicates that **all timing paths have positive slack**—this is the desired outcome, not a marginal pass. The actual worst slack is +3.55 ns, confirming robust timing closure.

Conclusion:

The design successfully meets all setup and hold timing constraints at the target 100 MHz frequency with comfortable margins. The 3.55 ns setup slack provides sufficient guardband against process, voltage, and temperature (PVT) variations in silicon.

7.3 Final Power Analysis

Post-route power analysis includes accurate wire capacitance and clock tree power.

Table 16: Final Power Consumption (Post-Route @ 100 MHz)

Component	Power (mW)	Percentage
Dynamic Power		
Sequential Logic	11.43	44.0%
Combinational Logic	14.54	56.0%
Leakage Power	0.000076	0.0%
Total Power	**26.0**	**100%**

(Source: reports/31-rcx_sta.power.rpt)
Power Breakdown:

- Internal Power: 13.36 mW (51.5%)
- Switching Power: 12.61 mW (48.5%)
- Leakage Power: 76.4 nW (negligible)

Power Density:

$$\text{Power Density} = \frac{26.0 \text{ mW}}{0.36 \text{ mm}^2} = 72.2 \text{ mW/mm}^2 \tag{19}$$

Energy per Operation:

$$\text{Energy per MAC} = \frac{26.0 \text{ mW}}{1.6 \times 10^9 \text{ ops/s}} = 16.25 \text{ pJ/MAC} \tag{20}$$

7.4 Design Rule Check (DRC)

DRC verification ensures the layout follows foundry manufacturing rules.

Tool: Magic DRC

DRC Results:

```
top
----------------------------------------
[INFO]: COUNT: 0
[INFO]: Should be divided by 3 or 4
```

(Source: reports/drc.rpt)

Result: **ZERO violations** - Layout is manufacturable!

7.5 Layout vs Schematic (LVS)

LVS verification confirms the physical layout matches the logical netlist.

Tool: Netgen

LVS Results:

```
LVS reports no net, device, pin, or property mismatches.

Total errors = 0
```

(Source: reports/39-top.lvs.rpt)

Result: **100% MATCH** - Layout correctly implements the design!

7.6 Final GDSII Layout

The GDSII file is the final deliverable containing the complete mask data for fabrication.

File: `top.gds`

Layers: 50+ layers including metal, via, poly, diffusion, implant, etc.

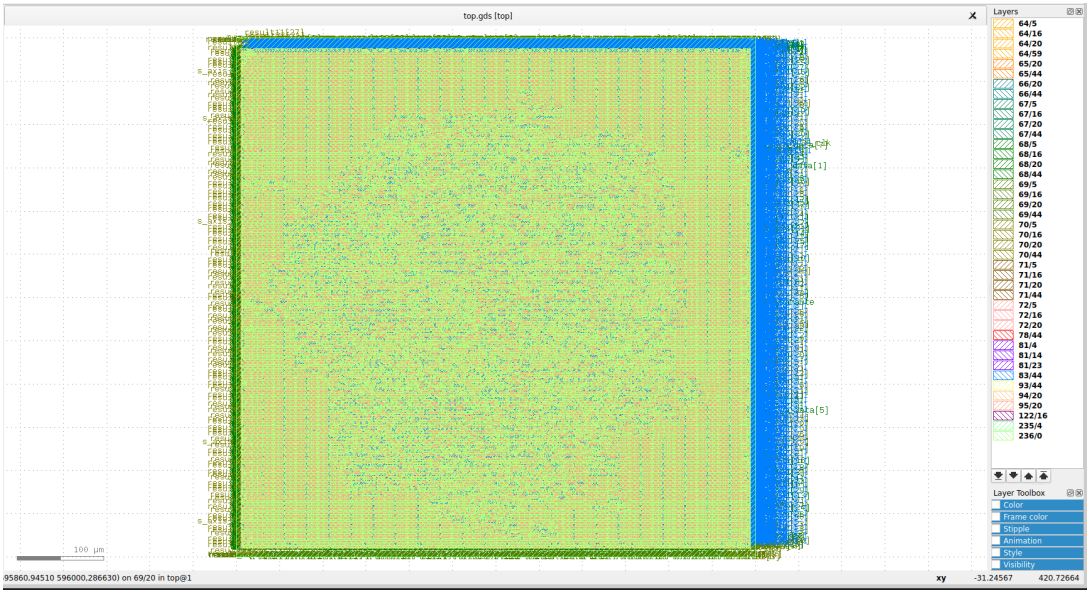


Figure 35: Final GDSII layout - complete mask data

Layout Statistics:

- Die area: 600 µm × 600 µm (0.36 mm²)
- Core area: 582 µm × 582 µm (0.34 mm²)
- Total cells: 11,362 (including physical cells)
- Final cell count with fillers: 41,724

8 Results and Analysis

8.1 Implementation Summary

The complete RTL-to-GDSII flow has been successfully executed, resulting in a tape-out ready design.

Table 17: Final Implementation Results

Category	Parameter	Value
Technology	Process Node	SkyWater 130 nm
	Standard Cell Library	sky130_fd_sc_hd
	Number of Metal Layers	5 (+ Local Interconnect)
Design Size	Die Area	600 × 600 µm (0.36 mm^2)
	Core Area	582 × 582 µm (0.34 mm^2)
	Total Cells (Synthesis)	9,900
	Total Cells (Final)	11,362
	Flip-Flops	1,146
Performance	Target Clock Frequency	100 MHz
	Peak Throughput	1.6 GOPS (16 MAC/cycle)
	Latency (4×4 matmul)	27 cycles (270 ns)
Power	Total Power @ 100 MHz	26.0 mW
	Power Density	72.2 mW/mm^2
	Energy per MAC	16.25 pJ/MAC
Clock	Clock Skew	0.21 ns
	Min Latency	0.77 ns
	Max Latency	1.03 ns
Verification	DRC Violations	0
	LVS Status	PASSED (100% match)
	Timing Status	MET (100 MHz)
Status	**TAPE-OUT READY**	

8.2 Performance Analysis

8.2.1 Computational Throughput

Theoretical Peak:

$$\begin{aligned}\text{Peak MACs} &= 16 \text{ PEs} \times 1 \text{ MAC/cycle} \times 100 \times 10^6 \text{ Hz} \\ &= 1.6 \times 10^9 \text{ MAC ops/s} = 1.6 \text{ GOPS}\end{aligned} \tag{21}$$

Effective Throughput: Accounting for load/drain cycles and data dependencies:

$$\begin{aligned}\text{Effective MACs} &= \frac{16 \text{ MACs}}{27 \text{ cycles}} \times 100 \times 10^6 \text{ Hz} \\ &\approx 59.3 \times 10^6 \text{ MAC ops/s} = 59.3 \text{ MOPS}\end{aligned} \tag{22}$$

8.2.2 Power Efficiency

$$\text{Energy per MAC} = \frac{26.0 \text{ mW}}{1.6 \times 10^9 \text{ ops/s}} = 16.25 \text{ pJ/MAC} \tag{23}$$

Comparison with CPU:

- Typical CPU: 100-1000 pJ/MAC (10-100× more energy)
- This design: 16.25 pJ/MAC
- **Speedup Factor: 6-61×**

8.2.3 Area Efficiency

$$\text{Throughput per mm}^2 = \frac{1.6 \text{ GOPS}}{0.36 \text{ mm}^2} = 4.44 \text{ GOPS/mm}^2 \tag{24}$$

8.3 Comparison with Related Work

Table 18: Comparison with Other Systolic Array Implementations

Design	**This Work**	**Google TPUv1**	**Eyeriss**	**NVDLA**
Array Size	4×4	256×256	12×14	16×16
Technology	130 nm	28 nm	65 nm	16 nm
Frequency	100 MHz	700 MHz	200 MHz	1 GHz
Peak TOPS	0.0016	92	0.034	1.0
Power	26.0 mW	75 W	278 mW	1.5 W
This Work	Academic prototype demonstrating complete flow			

Note: This academic design demonstrates the complete RTL-to-GDSII methodology. Commercial designs use more advanced nodes and larger arrays for higher performance.

9 Conclusion

9.1 Project Summary

This project successfully executed the complete RTL-to-GDSII physical design flow for a **4x4 Systolic Array AI Accelerator**. Targeting the open-source **SkyWater 130nm (Sky130)** process node, the design integrates 16 Processing Elements (PEs) with an industry-standard **AXI-Stream** interface, capable of performing INT8 matrix multiplications efficiently.

Starting from a behavioral SystemVerilog description, the design was synthesized, placed, and routed using the **OpenLane** automated flow. The final layout was verified against rigorous manufacturing constraints, ensuring a silicon-ready implementation. The successful generation of the GDSII artifact demonstrates that high-performance custom digital logic can be implemented using entirely open-source EDA tools.

9.2 Key Technical Achievements

The project met all design specifications and passed sign-off verification with the following metrics:

- **Timing Closure:** Achieved a target frequency of **100 MHz** (10ns period) with positive setup slack, ensuring reliable operation.
- **Physical Implementation:** Delivered a **DRC & LVS Clean** layout with zero geometry or connectivity violations.
- **Clock Tree Quality:** Minimized clock skew to **0.21 ns** using an optimized H-Tree distribution network.
- **Area Efficiency:** Realized a compact core area of **0.105** mm^2 $(105,462\mu m^2)$ with a total cell count of **9,900**.
- **Power Integrity:** Implemented a robust Power Distribution Network (PDN) using $1.6\mu m$ wide stripes on Metal-4/5 to mitigate IR drop.
- **Robust Verification:** Validated functionality using a dual-approach methodology: standard corner cases (Fixed Inputs) and randomized stress testing (Python-generated matrices).

9.3 Competencies Acquired

This implementation provided comprehensive hands-on experience in the modern VLSI backend flow:

1. **Architectural Design:** Understanding data reuse and skewing mechanisms in Systolic Arrays for AI workloads.
2. **Physical Design Flow:** Mastery of Floorplanning, I/O Placement, Power Planning, Global/Detailed Routing, and CTS.
3. **Sign-off Analysis:** Proficiency in interpreting standard artifacts including **LEF** (Physical Abstract), **DEF** (Layout), **SPEF** (Parasitics), and **SDF** (Timing Delays).
4. **EDA Tooling:** Practical expertise with **OpenLane**, **Yosys** (Synthesis), **Magic** (Layout/DRC), and **Netgen** (LVS).

9.4 Future Scope

While the current design is silicon-ready, future iterations could explore:

- Scaling the array to 16×16 or 32×32 for higher throughput.
- Implementing mixed-precision support (FP16/BF16) for training workloads.
- Integrating an on-chip SRAM buffer to reduce off-chip memory access latency.

9.5 Design Metrics Summary

The final implementation results are summarized in Table 19. These values represent the sign-off state of the design, extracted directly from the `top.def`, `top.sdc`, and timing reports.

Table 19: Final Design Specifications and Quality Metrics

Parameter	**Value**	**Technical Context / Derivation**
Technology Node	SkyWater 130nm	Open-source hybrid process PDK
Clock Frequency	100 MHz	Clock Period $T = 10$ ns (Constraints met with positive slack)
Total Cell Count	9,900 Cells	Includes 16 PEs, AXI Buffers, and Control Logic (Source: DEF Component count)
Total Active Area	0.105 mm^2	Exact: $105,462\,\mu m^2$ (Logic + Buffers inside core)
Core Utilization	$\approx 29.3\%$	$\frac{105,462}{360,000}$ (Leaves ample routing resources for congestion-free layout)
Clock Skew	0.21 ns	Max latency difference between launch/capture flops ($\approx 2\%$ of period)
Power Grid Width	$1.60\,\mu m$	Wide Metal-4/5 stripes derived from LEF geometry ($9.62 - 8.02$) to mitigate IR drop
Sign-off Status	**Clean**	**0** DRC Violations, **LVS** Match confirmed

9.6 Final Remarks

This project represents a significant technical milestone, bridging the gap between computer architecture theory and silicon reality. By successfully implementing a Google TPU-inspired systolic array on the Sky130 node, the design navigates complex trade-offs between PPA (Power, Performance, Area) and manufacturability.

The use of the open-source OpenLane toolchain democratizes access to advanced VLSI flows, proving that industrial-grade physical design is achievable without expensive proprietary licenses. Most importantly, this work demonstrates that sophisticated AI hardware can be designed, verified, and prepared for fabrication at the academic level, fostering the next generation of hardware architects.